

7 Linear Algebra

This chapter is co-authored with Zico Kolter.

7.1 Introduction

Linear algebra is the study of matrices and vectors. In this chapter, we summarize the key material that we will need throughout the book. Much more information can be found in other sources, such as [Str09; Ips09; Kle13; Mol04; TB97; Axl15; Tho17; Agg20].

7.1.1 Notation

In this section, we define some notation.

7.1.1.1 Vectors

A **vector** $\mathbf{x} \in \mathbb{R}^n$ is a list of n numbers, usually written as a **column vector**

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (7.1)$$

The vector of all ones is denoted $\mathbf{1}$. The vector of all zeros is denoted $\mathbf{0}$.

The **unit vector** $\mathbf{e}_i$ is a vector of all 0's, except entry i , which has value 1:

$$\mathbf{e}_i = (0, \dots, 0, 1, 0, \dots, 0) \quad (7.2)$$

This is also called a **one-hot vector**.

7.1.1.2 Matrices

A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ with m rows and n columns is a 2d array of numbers, arranged as follows:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}. \quad (7.3)$$

If $m = n$, the matrix is said to be **square**.

We use the notation A_{ij} or $A_{i,j}$ to denote the entry of $\mathbf{A}$ in the i th row and j th column. We use the notation $\mathbf{A}_{i,:}$ to denote the i 'th row and $\mathbf{A}_{:,j}$ to denote the j 'th column. We treat all vectors as column vectors by default (so $\mathbf{A}_{i,:}$ is viewed as a column vector with n entries). We use bold upper case letters to denote matrices, bold lower case letters to denote vectors, and non-bold letters to denote scalars.

We can view a matrix as a set of columns stacked along the horizontal axis:

$$\mathbf{A} = \left[\begin{array}{c|c|c|c} | & | & & | \\ \mathbf{A}_{:,1} & \mathbf{A}_{:,2} & \cdots & \mathbf{A}_{:,n} \\ | & | & & | \end{array} \right]. \quad (7.4)$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{:,1}, \mathbf{A}_{:,2}, \dots, \mathbf{A}_{:,n}] \quad (7.5)$$

We can also view a matrix as a set of rows stacked along the vertical axis:

$$\mathbf{A} = \left[\begin{array}{c} - & \mathbf{A}_{1,:}^T & - \\ - & \mathbf{A}_{2,:}^T & - \\ & \vdots & \\ - & \mathbf{A}_{m,:}^T & - \end{array} \right]. \quad (7.6)$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{1,:}; \mathbf{A}_{2,:}; \dots; \mathbf{A}_{m,:}] \quad (7.7)$$

(Note the use of a semicolon.)

The **transpose** of a matrix results from “flipping” the rows and columns. Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, its transpose, written $\mathbf{A}^T \in \mathbb{R}^{n \times m}$, is defined as

$$(\mathbf{A}^T)_{ij} = A_{ji} \quad (7.8)$$

The following properties of transposes are easily verified:

$$(\mathbf{A}^T)^T = \mathbf{A} \quad (7.9)$$

$$(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T \quad (7.10)$$

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T \quad (7.11)$$

If a square matrix satisfies $\mathbf{A} = \mathbf{A}^T$, it is called **symmetric**. We denote the set of all symmetric matrices of size n as $\mathbb{S}^n$.

7.1.1.3 Tensors

A **tensor** (in machine learning terminology) is just a generalization of a 2d array to more than 2 dimensions, as illustrated in Figure 7.1. For example, the entries of a 3d tensor are denoted by A_{ijk} .

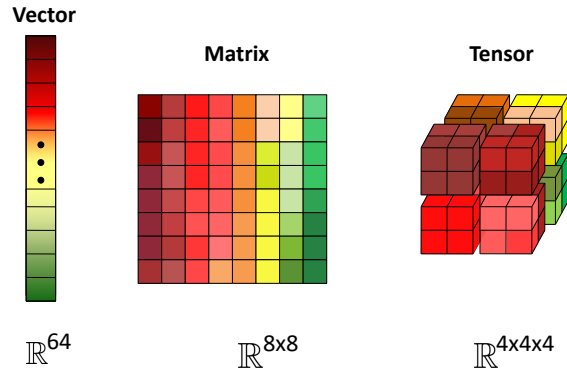


Figure 7.1: Illustration of a 1d vector, 2d matrix, and 3d tensor. The colors are used to represent individual entries of the vector; this list of numbers can also be stored in a 2d matrix, as shown. (In this example, the matrix is layed out in column-major order, which is the opposite of that used by Python.) We can also reshape the vector into a 3d tensor, as shown.



Figure 7.2: Illustration of (a) row-major vs (b) column-major order. From https://commons.wikimedia.org/wiki/File:Row_and_column_major_order.svg. Used with kind permission of Wikipedia author Cmglee.

The number of dimensions is known as the **order** or **rank** of the tensor.¹ In mathematics, tensors can be viewed as a way to define multilinear maps, just as matrices can be used to define linear functions, although we will not need to use this interpretation.

We can **reshape** a matrix into a vector by stacking its columns on top of each other, as shown in Figure 7.1. This is denoted by

$$\text{vec}(\mathbf{A}) = [\mathbf{A}_{:,1}; \cdots; \mathbf{A}_{:,n}] \in \mathbb{R}^{mn \times 1} \quad (7.12)$$

Conversely, we can reshape a vector into a matrix. There are two choices for how to do this, known as **row-major order** (used by languages such as Python and C++) and **column-major order** (used by languages such as Julia, Matlab, R and Fortran). See Figure 7.2 for an illustration of the difference.

1. Note, however, that the rank of a 2d matrix is a different concept, as discussed in Section 7.1.4.3.

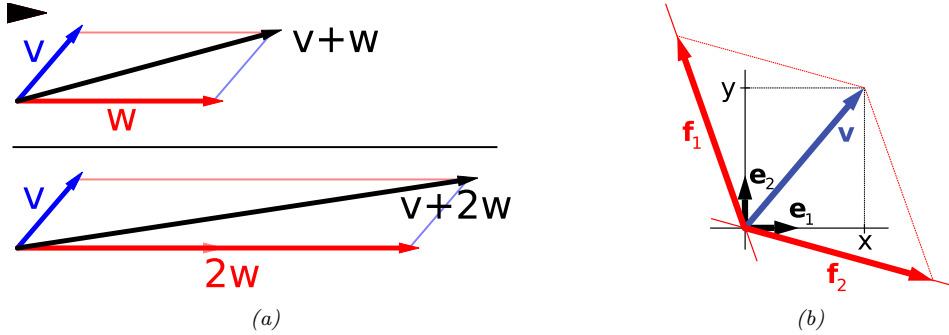


Figure 7.3: (a) Top: A vector $\mathbf{v}$ (blue) is added to another vector $\mathbf{w}$ (red). Bottom: $\mathbf{w}$ is stretched by a factor of 2, yielding the sum $\mathbf{v} + 2\mathbf{w}$. From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikipedia author IkamusumeFan (b) A vector $\mathbf{v}$ in $\mathbb{R}^2$ (blue) expressed in terms of different bases: using the standard basis of $\mathbb{R}^2$, $\mathbf{v} = x\mathbf{e}_1 + y\mathbf{e}_2$ (black), and using a different, non-orthogonal basis: $\mathbf{v} = \mathbf{f}_1 + \mathbf{f}_2$ (red). From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikipedia author Jakob.scholbach.

7.1.2 Vector spaces

In this section, we discuss some fundamental concepts in linear algebra.

7.1.2.1 Vector addition and scaling

We can view a vector $\mathbf{x} \in \mathbb{R}^n$ as defining a point in n -dimensional Euclidean space. A **vector space** is a collection of such vectors, which can be added together, and scaled by **scalars** (1-dimensional numbers), in order to create new points. These operations are defined to operate elementwise, in the obvious way, namely $\mathbf{x} + \mathbf{y} = (x_1 + y_1, \dots, x_n + y_n)$ and $c\mathbf{x} = (cx_1, \dots, cx_n)$, where $c \in \mathbb{R}$. See Figure 7.3a for an illustration.

7.1.2.2 Linear independence, spans and basis sets

A set of vectors $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ is said to be **(linearly) independent** if no vector can be represented as a linear combination of the remaining vectors. Conversely, a vector which *can* be represented as a linear combination of the remaining vectors is said to be **(linearly) dependent**. For example, if

$$\mathbf{x}_n = \sum_{i=1}^{n-1} \alpha_i \mathbf{x}_i \quad (7.13)$$

for some $\{\alpha_1, \dots, \alpha_{n-1}\}$ then $\mathbf{x}_n$ is dependent on $\{\mathbf{x}_1, \dots, \mathbf{x}_{n-1}\}$; otherwise, it is independent of $\{\mathbf{x}_1, \dots, \mathbf{x}_{n-1}\}$.

The **span** of a set of vectors $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ is the set of all vectors that can be expressed as a linear combination of $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$. That is,

$$\text{span}(\{\mathbf{x}_1, \dots, \mathbf{x}_n\}) \triangleq \left\{ \mathbf{v} : \mathbf{v} = \sum_{i=1}^n \alpha_i \mathbf{x}_i, \alpha_i \in \mathbb{R} \right\}. \quad (7.14)$$

It can be shown that if $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ is a set of n linearly independent vectors, where each $\mathbf{x}_i \in \mathbb{R}^n$, then $\text{span}(\{\mathbf{x}_1, \dots, \mathbf{x}_n\}) = \mathbb{R}^n$. In other words, *any* vector $\mathbf{v} \in \mathbb{R}^n$ can be written as a linear combination of $\mathbf{x}_1$ through $\mathbf{x}_n$.

A **basis** $\mathcal{B}$ is a set of linearly independent vectors that spans the whole space, meaning that $\text{span}(\mathcal{B}) = \mathbb{R}^n$. There are often multiple bases to choose from, as illustrated in Figure 7.3b. The **standard basis** uses the **coordinate vectors** $\mathbf{e}_1 = (1, 0, \dots, 0)$, up to $\mathbf{e}_n = (0, 0, \dots, 0, 1)$. This lets us translate back and forth between viewing a vector in $\mathbb{R}^2$ as an either an “arrow in the plane”, rooted at the origin, or as an ordered list of numbers (corresponding to the coefficients for each basis vector).

7.1.2.3 Linear maps and matrices

A **linear map** or **linear transformation** is any function $f : \mathcal{V} \rightarrow \mathcal{W}$ such that $f(\mathbf{v} + \mathbf{w}) = f(\mathbf{v}) + f(\mathbf{w})$ and $f(a\mathbf{v}) = a f(\mathbf{v})$ for all $\mathbf{v}, \mathbf{w} \in \mathcal{V}$. Once the basis of $\mathcal{V}$ is chosen, a linear map $f : \mathcal{V} \rightarrow \mathcal{W}$ is completely determined by specifying the images of the basis vectors, because any element of $\mathcal{V}$ can be expressed uniquely as a linear combination of them.

Suppose $\mathcal{V} = \mathbb{R}^n$ and $\mathcal{W} = \mathbb{R}^m$. We can compute $f(\mathbf{v}_i) \in \mathbb{R}^m$ for each basis vector in $\mathcal{V}$, and store these along the columns of an $m \times n$ matrix $\mathbf{A}$. We can then compute $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^m$ for any $\mathbf{x} \in \mathbb{R}^n$ as follows:

$$\mathbf{y} = \left(\sum_{j=1}^n a_{1j}x_j, \dots, \sum_{j=1}^n a_{mj}x_j \right) \quad (7.15)$$

This corresponds to multiplying the vector $\mathbf{x}$ by the matrix $\mathbf{A}$:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \quad (7.16)$$

See Section 7.2 for more details.

If the function is invertible, we can write

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{y} \quad (7.17)$$

See Section 7.3 for details.

7.1.2.4 Range and nullspace of a matrix

Suppose we view a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as a set of n vectors in $\mathbb{R}^m$. The **range** (sometimes also called the **column space**) of this matrix is the span of the columns of $\mathbf{A}$. In other words,

$$\text{range}(\mathbf{A}) \triangleq \{\mathbf{v} \in \mathbb{R}^m : \mathbf{v} = \mathbf{A}\mathbf{x}, \mathbf{x} \in \mathbb{R}^n\}. \quad (7.18)$$

This can be thought of as the set of vectors that can be “reached” or “generated” by $\mathbf{A}$; it is a subspace of $\mathbb{R}^m$ whose dimensionality is given by the rank of $\mathbf{A}$ (see Section 7.1.4.3). The **nullspace** of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the set of all vectors that get mapped to the null vector when multiplied by $\mathbf{A}$, i.e.,

$$\text{nullspace}(\mathbf{A}) \triangleq \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\}. \quad (7.19)$$

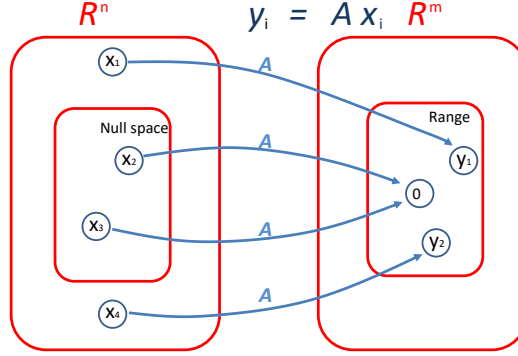


Figure 7.4: Visualization of the nullspace and range of an $m \times n$ matrix $\mathbf{A}$. Here $\mathbf{y}_1 = \mathbf{A}\mathbf{x}_1$ and $\mathbf{y}_2 = \mathbf{A}\mathbf{x}_4$, so $\mathbf{y}_1$ and $\mathbf{y}_2$ are in the range of $\mathbf{A}$ (are reachable from some $\mathbf{x}$). Also $\mathbf{A}\mathbf{x}_2 = \mathbf{0}$ and $\mathbf{A}\mathbf{x}_3 = \mathbf{0}$, so $\mathbf{x}_2$ and $\mathbf{x}_3$ are in the nullspace of $\mathbf{A}$ (get mapped to 0). We see that the range is often a subset of the input domain of the mapping.

The span of the rows of $\mathbf{A}$ is the complement to the nullspace of $\mathbf{A}$.

See Figure 7.4 for an illustration of the range and nullspace of a matrix. We shall discuss how to compute the range and nullspace of a matrix numerically in Section 7.5.4 below.

7.1.2.5 Linear projection

The **projection** of a vector $\mathbf{y} \in \mathbb{R}^m$ onto the span of $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ (here we assume $\mathbf{x}_i \in \mathbb{R}^m$) is the vector $\mathbf{v} \in \text{span}(\{\mathbf{x}_1, \dots, \mathbf{x}_n\})$, such that $\mathbf{v}$ is as close as possible to $\mathbf{y}$, as measured by the Euclidean norm $\|\mathbf{v} - \mathbf{y}\|_2$. We denote the projection as $\text{Proj}(\mathbf{y}; \{\mathbf{x}_1, \dots, \mathbf{x}_n\})$ and can define it formally as

$$\text{Proj}(\mathbf{y}; \{\mathbf{x}_1, \dots, \mathbf{x}_n\}) = \underset{\mathbf{v} \in \text{span}(\{\mathbf{x}_1, \dots, \mathbf{x}_n\})}{\text{argmin}} \|\mathbf{y} - \mathbf{v}\|_2. \quad (7.20)$$

Given a (full rank) matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$, we can define the projection of a vector $\mathbf{y} \in \mathbb{R}^m$ onto the range of $\mathbf{A}$ as follows:

$$\text{Proj}(\mathbf{y}; \mathbf{A}) = \underset{\mathbf{v} \in \mathcal{R}(\mathbf{A})}{\text{argmin}} \|\mathbf{v} - \mathbf{y}\|_2 = \mathbf{A}(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{y}. \quad (7.21)$$

These are the same as the normal equations from Section 11.2.2.2.

7.1.3 Norms of a vector and matrix

In this section, we discuss ways of measuring the “size” of a vector and matrix.

7.1.3.1 Vector norms

A **norm** of a vector $\|\mathbf{x}\|$ is, informally, a measure of the “length” of the vector. More formally, a norm is any function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ that satisfies 4 properties:

- For all $\mathbf{x} \in \mathbb{R}^n$, $f(\mathbf{x}) \geq 0$ (non-negativity).

- $f(\mathbf{x}) = 0$ if and only if $\mathbf{x} = \mathbf{0}$ (definiteness).
- For all $\mathbf{x} \in \mathbb{R}^n$, $t \in \mathbb{R}$, $f(t\mathbf{x}) = |t|f(\mathbf{x})$ (absolute value homogeneity).
- For all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (triangle inequality).

Consider the following common examples:

p-norm $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$, for $p \geq 1$.

2-norm $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$, also called Euclidean norm. Note that $\|\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{x}$.

1-norm $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$.

Max-norm $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

0-norm $\|\mathbf{x}\|_0 = \sum_{i=1}^n \mathbb{I}(|x_i| > 0)$. This is a **pseudo norm**, since it does not satisfy homogeneity. It counts the number of non-zero elements in $\mathbf{x}$. If we define $0^0 = 0$, we can write this as $\|\mathbf{x}\|_0 = \sum_{i=1}^n x_i^0$.

7.1.3.2 Matrix norms

Suppose we think of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as defining a linear function $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$. We define the **induced norm** of $\mathbf{A}$ as the maximum amount by which f can lengthen any unit-norm input:

$$\|\mathbf{A}\|_p = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_p}{\|\mathbf{x}\|_p} = \max_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|_p \quad (7.22)$$

Typically $p = 2$, in which case

$$\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})} = \max_i \sigma_i \quad (7.23)$$

where $\lambda_{\max}(\mathbf{M})$ is the largest eigenvalue of $\mathbf{M}$, and σ_i is the i 'th singular value.

The **nuclear norm**, also called the **trace norm**, is defined as

$$\|\mathbf{A}\|_* = \text{tr}(\sqrt{\mathbf{A}^\top \mathbf{A}}) = \sum_i \sigma_i \quad (7.24)$$

where $\sqrt{\mathbf{A}^\top \mathbf{A}}$ is the matrix square root. Since the singular values are always non-negative, we have

$$\|\mathbf{A}\|_* = \sum_i |\sigma_i| = \|\boldsymbol{\sigma}\|_1 \quad (7.25)$$

Using this as a regularizer encourages many singular values to become zero, resulting in a low rank matrix. More generally, we can define the **Schatten p-norm** as

$$\|\mathbf{A}\|_p = \left(\sum_i \sigma_i^p(\mathbf{A}) \right)^{1/p} \quad (7.26)$$

If we think of a matrix as a vector, we can define the matrix norm in terms of a vector norm, $\|\mathbf{A}\| = \|\text{vec}(\mathbf{A})\|$. If the vector norm is the 2-norm, the corresponding matrix norm is the **Frobenius norm**:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})} = \|\text{vec}(\mathbf{A})\|_2 \quad (7.27)$$

If $\mathbf{A}$ is expensive to evaluate, but $\mathbf{A}\mathbf{v}$ is cheap (for a random vector $\mathbf{v}$), we can create a stochastic approximation to the Frobenius norm by using the Hutchinson trace estimator from Equation (7.37) as follows:

$$\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A}) = \mathbb{E}[\mathbf{v}^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}] = \mathbb{E}[\|\mathbf{A}\mathbf{v}\|_2^2] \quad (7.28)$$

where $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

7.1.4 Properties of a matrix

In this section, we discuss various scalar properties of matrices.

7.1.4.1 Trace of a square matrix

The **trace** of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, denoted $\text{tr}(\mathbf{A})$, is the sum of diagonal elements in the matrix:

$$\text{tr}(\mathbf{A}) \triangleq \sum_{i=1}^n A_{ii}. \quad (7.29)$$

The trace has the following properties, where $c \in \mathbb{R}$ is a scalar, and $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ are square matrices:

$$\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}^\top) \quad (7.30)$$

$$\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B}) \quad (7.31)$$

$$\text{tr}(c\mathbf{A}) = c \text{tr}(\mathbf{A}) \quad (7.32)$$

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) \quad (7.33)$$

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \quad (7.34)$$

We also have the following important **cyclic permutation property**: For $\mathbf{A}, \mathbf{B}, \mathbf{C}$ such that $\mathbf{ABC}$ is square,

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB}) \quad (7.35)$$

From this, we can derive the **trace trick**, which rewrites the scalar inner product $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ as follows

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \text{tr}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = \text{tr}(\mathbf{x} \mathbf{x}^\top \mathbf{A}) \quad (7.36)$$

In some cases, it may be expensive to evaluate the matrix $\mathbf{A}$, but we may be able to cheaply evaluate matrix-vector products $\mathbf{A}\mathbf{v}$. Suppose $\mathbf{v}$ is a random vector such that $\mathbb{E}[\mathbf{v}\mathbf{v}^\top] = \mathbf{I}$. In this case, we can create a Monte Carlo approximation to $\text{tr}(\mathbf{A})$ using the following identity:

$$\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}\mathbb{E}[\mathbf{v}\mathbf{v}^\top]) = \mathbb{E}[\text{tr}(\mathbf{A}\mathbf{v}\mathbf{v}^\top)] = \mathbb{E}[\text{tr}(\mathbf{v}^\top\mathbf{A}\mathbf{v})] = \mathbb{E}[\mathbf{v}^\top\mathbf{A}\mathbf{v}] \quad (7.37)$$

This is called the **Hutchinson trace estimator** [Hut90].

7.1.4.2 Determinant of a square matrix

The **determinant** of a square matrix, denoted $\det(\mathbf{A})$ or $|\mathbf{A}|$, is a measure of how much it changes a unit volume when viewed as a linear transformation. (The formal definition is rather complex and is not needed here.)

The determinant operator satisfies these properties, where $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$

$$|\mathbf{A}| = |\mathbf{A}^\top| \quad (7.38)$$

$$|c\mathbf{A}| = c^n |\mathbf{A}| \quad (7.39)$$

$$|\mathbf{A}\mathbf{B}| = |\mathbf{A}||\mathbf{B}| \quad (7.40)$$

$$|\mathbf{A}| = 0 \text{ iff } \mathbf{A} \text{ is singular} \quad (7.41)$$

$$|\mathbf{A}^{-1}| = 1/|\mathbf{A}| \text{ if } \mathbf{A} \text{ is not singular} \quad (7.42)$$

$$|\mathbf{A}| = \prod_{i=1}^n \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \quad (7.43)$$

For a positive definite matrix $\mathbf{A}$, we can write $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is the lower triangular Cholesky decomposition. In this case, we have

$$\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{L}^\top) = \det(\mathbf{L})^2 \quad (7.44)$$

so

$$\log \det(\mathbf{A}) = 2 \log \det(\mathbf{L}) = 2 \log \prod_i L_{ii} = 2 \text{tr}(\log(\text{diag}(\mathbf{L}))) \quad (7.45)$$

7.1.4.3 Rank of a matrix

The **column rank** of a matrix $\mathbf{A}$ is the dimension of the space spanned by its columns, and the **row rank** is the dimension of the space spanned by its rows. It is a basic fact of linear algebra (that can be shown using the SVD, discussed in Section 7.5) that for any matrix $\mathbf{A}$, $\text{columnrank}(\mathbf{A}) = \text{rowrank}(\mathbf{A})$, and so this quantity is simply referred to as the **rank** of $\mathbf{A}$, denoted as $\text{rank}(\mathbf{A})$. The following are some basic properties of the rank:

- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\text{rank}(\mathbf{A}) \leq \min(m, n)$. If $\text{rank}(\mathbf{A}) = \min(m, n)$, then $\mathbf{A}$ is said to be **full rank**, otherwise it is called **rank deficient**.
- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^\top) = \text{rank}(\mathbf{A}^\top\mathbf{A}) = \text{rank}(\mathbf{A}\mathbf{A}^\top)$.
- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$, $\text{rank}(\mathbf{A}\mathbf{B}) \leq \min(\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B}))$.

- For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$, $\text{rank}(\mathbf{A} + \mathbf{B}) \leq \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B})$.

One can show that a square matrix is invertible iff it is full rank.

7.1.4.4 Condition numbers

The **condition number** of a matrix $\mathbf{A}$ is a measure of how numerically stable any computations involving $\mathbf{A}$ will be. It is defined as follows:

$$\kappa(\mathbf{A}) \triangleq \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \quad (7.46)$$

where $\|\mathbf{A}\|$ is the norm of the matrix. We can show that $\kappa(\mathbf{A}) \geq 1$. (The condition number depends on which norm we use; we will assume the ℓ_2 -norm unless stated otherwise.)

We say $\mathbf{A}$ is **well-conditioned** if $\kappa(\mathbf{A})$ is small (close to 1), and **ill-conditioned** if $\kappa(\mathbf{A})$ is large. A large condition number means $\mathbf{A}$ is nearly singular. This is a better measure of nearness to singularity than the size of the determinant. For example, suppose $\mathbf{A} = 0.1\mathbf{I}_{100 \times 100}$. Then $\det(\mathbf{A}) = 10^{-100}$, which suggests $\mathbf{A}$ is nearly singular, but $\kappa(\mathbf{A}) = 1$, which means $\mathbf{A}$ is well-conditioned, reflecting the fact that $\mathbf{A}\mathbf{x}$ simply scales the entries of $\mathbf{x}$ by 0.1.

To get a better understanding of condition numbers, consider the linear system of equations $\mathbf{A}\mathbf{x} = \mathbf{b}$. If $\mathbf{A}$ is non-singular, the unique solution is $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. Suppose we change $\mathbf{b}$ to $\mathbf{b} + \Delta\mathbf{b}$; what effect will that have on $\mathbf{x}$? The new solution must satisfy

$$\mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b} \quad (7.47)$$

where

$$\Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b} \quad (7.48)$$

We say that $\mathbf{A}$ is well-conditioned if a small $\Delta\mathbf{b}$ results in a small $\Delta\mathbf{x}$; otherwise we say that $\mathbf{A}$ is ill-conditioned.

For example, suppose

$$\mathbf{A} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 + 10^{-10} & 1 - 10^{-10} \end{pmatrix}, \quad \mathbf{A}^{-1} = \begin{pmatrix} 1 - 10^{10} & 10^{10} \\ 1 + 10^{10} & -10^{10} \end{pmatrix} \quad (7.49)$$

The solution for $\mathbf{b} = (1, 1)$ is $\mathbf{x} = (1, 1)$. If we change $\mathbf{b}$ by $\Delta\mathbf{b}$, the solution changes to

$$\Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b} = \begin{pmatrix} \Delta b_1 - 10^{10}(\Delta b_1 - \Delta b_2) \\ \Delta b_1 + 10^{10}(\Delta b_1 - \Delta b_2) \end{pmatrix} \quad (7.50)$$

So a small change in $\mathbf{b}$ can lead to an extremely large change in $\mathbf{x}$, because $\mathbf{A}$ is ill-conditioned ($\kappa(\mathbf{A}) = 2 \times 10^{10}$).

In the case of the ℓ_2 -norm, the condition number is equal to the ratio of the largest to smallest singular values (defined in Section 7.5); furthermore, the singular values of $\mathbf{A}$ are the square roots of the eigenvalues of $\mathbf{A}^T\mathbf{A}$, and so

$$\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min} = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}} \quad (7.51)$$

We can gain further insight into condition numbers by considering a quadratic objective function $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$. If we plot the level set of this function, it will be elliptical, as shown in Section 7.4.4. As we increase the condition number of $\mathbf{A}$, the ellipses become more and more elongated along certain directions, corresponding to a very narrow valley in function space. If $\kappa = 1$ (the minimum possible value), the level set will be circular.

7.1.5 Special types of matrices

In this section, we will list some common kinds of matrices with various forms of structure.

7.1.5.1 Diagonal matrix

A **diagonal matrix** is a matrix where all non-diagonal elements are 0. This is typically denoted $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_n)$, with

$$\mathbf{D} = \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix} \quad (7.52)$$

The **identity matrix**, denoted $\mathbf{I} \in \mathbb{R}^{n \times n}$, is a square matrix with ones on the diagonal and zeros everywhere else, $\mathbf{I} = \text{diag}(1, 1, \dots, 1)$. It has the property that for all $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\mathbf{A}\mathbf{I} = \mathbf{A} = \mathbf{I}\mathbf{A} \quad (7.53)$$

where the size of $\mathbf{I}$ is determined by the dimensions of $\mathbf{A}$ so that matrix multiplication is possible.

We can extract the diagonal vector from a matrix using $\mathbf{d} = \text{diag}(\mathbf{D})$. We can convert a vector into a diagonal matrix by writing $\mathbf{D} = \text{diag}(\mathbf{d})$.

A **block diagonal matrix** is one which contains matrices on its main diagonal, and is 0 everywhere else, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix} \quad (7.54)$$

A **band-diagonal matrix** only has non-zero entries along the diagonal, and on k sides of the diagonal, where k is the bandwidth. For example, a **tridiagonal** 6×6 matrix looks like this:

$$\begin{bmatrix} A_{11} & A_{12} & 0 & \cdots & \cdots & 0 \\ A_{21} & A_{22} & A_{23} & \ddots & \ddots & \vdots \\ 0 & A_{32} & A_{33} & A_{34} & \ddots & \vdots \\ \vdots & \ddots & A_{43} & A_{44} & A_{45} & 0 \\ \vdots & \ddots & \ddots & A_{54} & A_{55} & A_{56} \\ 0 & \cdots & \cdots & 0 & A_{65} & A_{66} \end{bmatrix} \quad (7.55)$$

7.1.5.2 Triangular matrices

An **upper triangular matrix** only has non-zero entries on and above the diagonal. A **lower triangular matrix** only has non-zero entries on and below the diagonal.

Triangular matrices have the useful property that the diagonal entries of $\mathbf{A}$ are the eigenvalues of $\mathbf{A}$, and hence the determinant is the product of diagonal entries: $\det(\mathbf{A}) = \prod_i A_{ii}$.

7.1.5.3 Positive definite matrices

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, the scalar value $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ is called a **quadratic form**. Written explicitly, we see that

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n A_{ij} x_i x_j . \quad (7.56)$$

Note that,

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = (\mathbf{x}^\top \mathbf{A} \mathbf{x})^\top = \mathbf{x}^\top \mathbf{A}^\top \mathbf{x} = \mathbf{x}^\top \left(\frac{1}{2} \mathbf{A} + \frac{1}{2} \mathbf{A}^\top \right) \mathbf{x} \quad (7.57)$$

For this reason, we often implicitly assume that the matrices appearing in a quadratic form are symmetric.

We give the following definitions:

- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **positive definite** iff for all non-zero vectors $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$. This is usually denoted $\mathbf{A} \succ 0$ (or just $\mathbf{A} > 0$). If it is possible that $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 0$, we say the matrix is **positive semidefinite** or **psd**. We denote the set of all positive definite matrices by $\mathbb{S}_{++}^n$.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **negative definite**, denoted $\mathbf{A} \prec 0$ (or just $\mathbf{A} < 0$) iff for all non-zero $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{A} \mathbf{x} < 0$. If it is possible that $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 0$, we say the matrix is **negative semidefinite**.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **indefinite**, if it is neither positive semidefinite nor negative semidefinite — i.e., if there exists $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$ such that $\mathbf{x}_1^\top \mathbf{A} \mathbf{x}_1 > 0$ and $\mathbf{x}_2^\top \mathbf{A} \mathbf{x}_2 < 0$.

It should be obvious that if $\mathbf{A}$ is positive definite, then $-\mathbf{A}$ is negative definite and vice versa. Likewise, if $\mathbf{A}$ is positive semidefinite then $-\mathbf{A}$ is negative semidefinite and vice versa. If $\mathbf{A}$ is indefinite, then so is $-\mathbf{A}$. It can also be shown that positive definite and negative definite matrices are always invertible.

In Section 7.4.3.1, we show that a symmetric matrix is positive definite iff its eigenvalues are positive. Note that if all elements of $\mathbf{A}$ are positive, it does not mean $\mathbf{A}$ is necessarily positive definite. For example, $\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 3 & 2 \end{pmatrix}$ is not positive definite. Conversely, a positive definite matrix can have

negative entries e.g., $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$

A sufficient condition for a (real, symmetric) matrix to be positive definite is that it is **diagonally dominant**, i.e., if in every row of the matrix, the magnitude of the diagonal entry in that row is

larger than the sum of the magnitudes of all the other (non-diagonal) entries in that row. More precisely,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i \quad (7.58)$$

In 2d, any real, symmetric 2×2 matrix $\begin{pmatrix} a & b \\ b & d \end{pmatrix}$ is positive definite iff $a > 0$, $d > 0$ and $ad > b^2$.

Finally, there is one type of positive definite matrix that comes up frequently, and so deserves some special mention. Given any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (not necessarily symmetric or even square), the **Gram matrix** $\mathbf{G} = \mathbf{A}^T \mathbf{A}$ is always positive semidefinite. Further, if $m \geq n$ (and we assume for convenience that $\mathbf{A}$ is full rank), then $\mathbf{G} = \mathbf{A}^T \mathbf{A}$ is positive definite.

7.1.5.4 Orthogonal matrices

Two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ are **orthogonal** if $\mathbf{x}^T \mathbf{y} = 0$. A vector $\mathbf{x} \in \mathbb{R}^n$ is **normalized** if $\|\mathbf{x}\|_2 = 1$. A set of vectors that is pairwise orthogonal and normalized is called **orthonormal**. A square matrix $\mathbf{U} \in \mathbb{R}^{n \times n}$ is **orthogonal** if all its columns are orthonormal. (Note the different meaning of the term orthogonal when talking about vectors versus matrices.) If the entries of $\mathbf{U}$ are complex valued, we use the term **unitary** instead of orthogonal.

It follows immediately from the definition of orthogonality and normality that $\mathbf{U}$ is orthogonal iff

$$\mathbf{U}^T \mathbf{U} = \mathbf{I} = \mathbf{U} \mathbf{U}^T. \quad (7.59)$$

In other words, the inverse of an orthogonal matrix is its transpose. Note that if $\mathbf{U}$ is not square — i.e., $\mathbf{U} \in \mathbb{R}^{m \times n}$, $n < m$ — but its columns are still orthonormal, then $\mathbf{U}^T \mathbf{U} = \mathbf{I}$, but $\mathbf{U} \mathbf{U}^T \neq \mathbf{I}$. We generally only use the term orthogonal to describe the previous case, where $\mathbf{U}$ is square.

An example of an orthogonal matrix is a **rotation matrix** (see Exercise 7.1). For example, a rotation in 3d by angle α about the z axis is given by

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (7.60)$$

If $\alpha = 45^\circ$, this becomes

$$\mathbf{R}(45) = \begin{pmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (7.61)$$

where $\frac{1}{\sqrt{2}} = 0.7071$. We see that $\mathbf{R}(-\alpha) = \mathbf{R}(\alpha)^{-1} = \mathbf{R}(\alpha)^T$, so this is an orthogonal matrix.

One nice property of orthogonal matrices is that operating on a vector with an orthogonal matrix will not change its Euclidean norm, i.e.,

$$\|\mathbf{U}\mathbf{x}\|_2 = \|\mathbf{x}\|_2 \quad (7.62)$$

for any nonzero $\mathbf{x} \in \mathbb{R}^n$, and orthogonal $\mathbf{U} \in \mathbb{R}^{n \times n}$.

Similarly, one can show that the angle between two vectors is preserved after they are transformed by an orthogonal matrix. The cosine of the angle between $\mathbf{x}$ and $\mathbf{y}$ is given by

$$\cos(\alpha(\mathbf{x}, \mathbf{y})) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (7.63)$$

so

$$\cos(\alpha(\mathbf{U}\mathbf{x}, \mathbf{U}\mathbf{y})) = \frac{(\mathbf{U}\mathbf{x})^\top (\mathbf{U}\mathbf{y})}{\|\mathbf{U}\mathbf{x}\| \|\mathbf{U}\mathbf{y}\|} = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \cos(\alpha(\mathbf{x}, \mathbf{y})) \quad (7.64)$$

In summary, transformations by orthogonal matrices are generalizations of rotations (if $\det(\mathbf{U}) = 1$) and reflections (if $\det(\mathbf{U}) = -1$), since they preserve lengths and angles.

Note that there is a technique called Gram Schmidt orthogonalization which is a way to make any square matrix orthogonal, but we will not cover it here.

7.2 Matrix multiplication

The product of two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is the matrix

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}, \quad (7.65)$$

where

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}. \quad (7.66)$$

Note that in order for the matrix product to exist, the number of columns in $\mathbf{A}$ must equal the number of rows in $\mathbf{B}$.

Matrix multiplication generally takes $O(mnp)$ time, although faster methods exist. In addition, specialized hardware, such as GPUs and TPUs, can be leveraged to speed up matrix multiplication significantly, by performing operations across the rows (or columns) in parallel.

It is useful to know a few basic properties of matrix multiplication:

- Matrix multiplication is **associative**: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
- Matrix multiplication is **distributive**: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.
- Matrix multiplication is, in general, *not* **commutative**; that is, it can be the case that $\mathbf{AB} \neq \mathbf{BA}$.

(In each of the above cases, we are assuming that the dimensions match.)

There are many important special cases of matrix multiplication, as we discuss below.

7.2.1 Vector–vector products

Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the quantity $\mathbf{x}^\top \mathbf{y}$, called the **inner product**, **dot product** or **scalar product** of the vectors, is a real number given by

$$\langle \mathbf{x}, \mathbf{y} \rangle \triangleq \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i. \quad (7.67)$$

Note that it is always the case that $\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}$.

Given vectors $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$ (they no longer have to be the same size), $\mathbf{x}\mathbf{y}^\top$ is called the **outer product** of the vectors. It is a matrix whose entries are given by $(\mathbf{x}\mathbf{y}^\top)_{ij} = x_i y_j$, i.e.,

$$\mathbf{x}\mathbf{y}^\top \in \mathbb{R}^{m \times n} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_m y_1 & x_m y_2 & \cdots & x_m y_n \end{bmatrix}. \quad (7.68)$$

7.2.2 Matrix–vector products

Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, their product is a vector $\mathbf{y} = \mathbf{A}\mathbf{x} \in \mathbb{R}^m$. There are a couple of ways of looking at matrix-vector multiplication, and we will look at them both.

If we write $\mathbf{A}$ by rows, then we can express $\mathbf{y} = \mathbf{A}\mathbf{x}$ as follows:

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} - & \mathbf{a}_1^\top & - \\ - & \mathbf{a}_2^\top & - \\ & \vdots & \\ - & \mathbf{a}_m^\top & - \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}. \quad (7.69)$$

In other words, the i th entry of $\mathbf{y}$ is equal to the inner product of the i th row of $\mathbf{A}$ and $\mathbf{x}$, $y_i = \mathbf{a}_i^\top \mathbf{x}$.

Alternatively, let's write $\mathbf{A}$ in column form. In this case we see that

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{a}_1 & \mathbf{a}_2 & & \mathbf{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} | \\ \mathbf{a}_1 \\ | \end{bmatrix} x_1 + \begin{bmatrix} | \\ \mathbf{a}_2 \\ | \end{bmatrix} x_2 + \cdots + \begin{bmatrix} | \\ \mathbf{a}_n \\ | \end{bmatrix} x_n. \quad (7.70)$$

In other words, $\mathbf{y}$ is a **linear combination** of the *columns* of $\mathbf{A}$, where the coefficients of the linear combination are given by the entries of $\mathbf{x}$. We can view the columns of $\mathbf{A}$ as a set of **basis vectors** defining a **linear subspace**. We can construct vectors in this subspace by taking linear combinations of the basis vectors. See Section 7.1.2 for details.

7.2.3 Matrix–matrix products

Below we look at four different (but, of course, equivalent) ways of viewing the matrix-matrix multiplication $\mathbf{C} = \mathbf{AB}$.

First we can view matrix-matrix multiplication as a set of vector-vector products. The most obvious viewpoint, which follows immediately from the definition, is that the i, j entry of $\mathbf{C}$ is equal to the inner product of the i th row of $\mathbf{A}$ and the j th column of $\mathbf{B}$. Symbolically, this looks like the following,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} - & \mathbf{a}_1^\top & - \\ - & \mathbf{a}_2^\top & - \\ & \vdots & \\ - & \mathbf{a}_m^\top & - \end{bmatrix} \begin{bmatrix} | & | & \cdots & | \\ \mathbf{b}_1 & \mathbf{b}_2 & & \mathbf{b}_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{b}_1 & \mathbf{a}_1^\top \mathbf{b}_2 & \cdots & \mathbf{a}_1^\top \mathbf{b}_p \\ \mathbf{a}_2^\top \mathbf{b}_1 & \mathbf{a}_2^\top \mathbf{b}_2 & \cdots & \mathbf{a}_2^\top \mathbf{b}_p \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_m^\top \mathbf{b}_1 & \mathbf{a}_m^\top \mathbf{b}_2 & \cdots & \mathbf{a}_m^\top \mathbf{b}_p \end{bmatrix}. \quad (7.71)$$

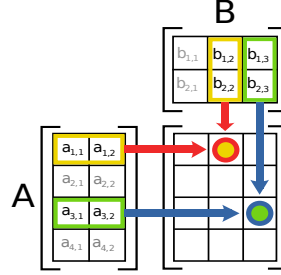


Figure 7.5: Illustration of matrix multiplication. From https://en.wikipedia.org/wiki/Matrix_multiplication. Used with kind permission of Wikipedia author Bilou.

Remember that since $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, $\mathbf{a}_i \in \mathbb{R}^n$ and $\mathbf{b}_j \in \mathbb{R}^n$, so these inner products all make sense. This is the most “natural” representation when we represent $\mathbf{A}$ by rows and $\mathbf{B}$ by columns. See Figure 7.5 for an illustration.

Alternatively, we can represent $\mathbf{A}$ by columns, and $\mathbf{B}$ by rows, which leads to the interpretation of $\mathbf{AB}$ as a sum of outer products. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} - & \mathbf{b}_1^\top & - \\ - & \mathbf{b}_2^\top & - \\ \vdots & \vdots & \\ - & \mathbf{b}_n^\top & - \end{bmatrix} = \sum_{i=1}^n \mathbf{a}_i \mathbf{b}_i^\top. \quad (7.72)$$

Put another way, $\mathbf{AB}$ is equal to the sum, over all i , of the outer product of the i th column of $\mathbf{A}$ and the i th row of $\mathbf{B}$. Since, in this case, $\mathbf{a}_i \in \mathbb{R}^m$ and $\mathbf{b}_i \in \mathbb{R}^p$, the dimension of the outer product $\mathbf{a}_i \mathbf{b}_i^\top$ is $m \times p$, which coincides with the dimension of $\mathbf{C}$.

We can also view matrix-matrix multiplication as a set of matrix-vector products. Specifically, if we represent $\mathbf{B}$ by columns, we can view the columns of $\mathbf{C}$ as matrix-vector products between $\mathbf{A}$ and the columns of $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \mathbf{A} \begin{bmatrix} | & | & \cdots & | \\ \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_p \\ | & | & \cdots & | \end{bmatrix} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{Ab}_1 & \mathbf{Ab}_2 & \cdots & \mathbf{Ab}_p \\ | & | & \cdots & | \end{bmatrix}. \quad (7.73)$$

Here the i th column of $\mathbf{C}$ is given by the matrix-vector product with the vector on the right, $\mathbf{c}_i = \mathbf{Ab}_i$. These matrix-vector products can in turn be interpreted using both viewpoints given in the previous subsection.

Finally, we have the analogous viewpoint, where we represent $\mathbf{A}$ by rows, and view the rows of $\mathbf{C}$ as the matrix-vector product between the rows of $\mathbf{A}$ and the matrix $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} - & \mathbf{a}_1^\top & - \\ - & \mathbf{a}_2^\top & - \\ \vdots & \vdots & \\ - & \mathbf{a}_m^\top & - \end{bmatrix} \mathbf{B} = \begin{bmatrix} - & \mathbf{a}_1^\top \mathbf{B} & - \\ - & \mathbf{a}_2^\top \mathbf{B} & - \\ \vdots & \vdots & \\ - & \mathbf{a}_m^\top \mathbf{B} & - \end{bmatrix}. \quad (7.74)$$

Here the i th row of $\mathbf{C}$ is given by the matrix-vector product with the vector on the left, $\mathbf{c}_i^\top = \mathbf{a}_i^\top \mathbf{B}$.

It may seem like overkill to dissect matrix multiplication to such a large degree, especially when all these viewpoints follow immediately from the initial definition we gave (in about a line of math) at the beginning of this section. However, virtually all of linear algebra deals with matrix multiplications of some kind, and it is worthwhile to spend some time trying to develop an intuitive understanding of the viewpoints presented here.

Finally, a word on notation. We write $\mathbf{A}^2$ as shorthand for $\mathbf{A}\mathbf{A}$, which is the matrix product. To denote elementwise squaring of the elements of a matrix, we write $\mathbf{A}^{\odot 2} = [A_{ij}^2]$. (If $\mathbf{A}$ is diagonal, then $\mathbf{A}^2 = \mathbf{A}^{\odot 2}$.)

We can also define the inverse of $\mathbf{A}^2$ using the **matrix square root**: we say $\mathbf{A} = \sqrt{\mathbf{M}}$ if $\mathbf{A}^2 = \mathbf{M}$. To denote elementwise square root of the elements of a matrix, we write $[\sqrt{M_{ij}}]$.

7.2.4 Application: manipulating data matrices

As an application of the above results, consider the case where $\mathbf{X}$ is the $N \times D$ design matrix, whose rows are the data cases. There are various common preprocessing operations that we apply to this matrix, which we summarize below. (Writing these operations in matrix form is useful because it is notationally compact, and it allows us to implement the methods quickly using fast matrix code.)

7.2.4.1 Summing slices of the matrix

Suppose $\mathbf{X}$ is an $N \times D$ matrix. We can sum across the rows by premultiplying by a $1 \times N$ matrix of ones to create a $1 \times D$ matrix:

$$\mathbf{1}_N^\top \mathbf{X} = (\sum_n x_{n1} \quad \cdots \quad \sum_n x_{nD}) \quad (7.75)$$

Hence the mean of the data vectors is given by

$$\bar{\mathbf{x}}^\top = \frac{1}{N} \mathbf{1}_N^\top \mathbf{X} \quad (7.76)$$

We can sum across the columns by postmultiplying by a $D \times 1$ matrix of ones to create a $N \times 1$ matrix:

$$\mathbf{X} \mathbf{1}_D = \begin{pmatrix} \sum_d x_{1d} \\ \vdots \\ \sum_d x_{Nd} \end{pmatrix} \quad (7.77)$$

We can sum all entries in a matrix by pre and post multiplying by a vector of 1s:

$$\mathbf{1}_N^\top \mathbf{X} \mathbf{1}_D = \sum_{ij} X_{ij} \quad (7.78)$$

Hence the overall mean is given by

$$\bar{x} = \frac{1}{ND} \mathbf{1}_N^\top \mathbf{X} \mathbf{1}_D \quad (7.79)$$

7.2.4.2 Scaling rows and columns of a matrix

We often want to scale rows or columns of a data matrix (e.g., to standardize them). We now show how to write this in matrix notation.

If we pre-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\mathbf{s})$, where $\mathbf{s}$ is an N -vector, then we just scale each row of $\mathbf{X}$ by the corresponding scale factor in $\mathbf{s}$:

$$\text{diag}(\mathbf{s})\mathbf{X} = \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_N \end{pmatrix} \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_1 x_{1,D} \\ & \ddots & \\ s_N x_{N,1} & \cdots & s_N x_{N,D} \end{pmatrix} \quad (7.80)$$

If we post-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\mathbf{s})$, where $\mathbf{s}$ is a D -vector, then we just scale each column of $\mathbf{X}$ by the corresponding element in $\mathbf{s}$.

$$\mathbf{X}\text{diag}(\mathbf{s}) = \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_D \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_D x_{1,D} \\ & \ddots & \\ s_1 x_{N,1} & \cdots & s_D x_{N,D} \end{pmatrix} \quad (7.81)$$

Thus we can rewrite the standardization operation from Section 10.2.8 in matrix form as follows:

$$\text{standardize}(\mathbf{X}) = (\mathbf{X} - \mathbf{1}_N \boldsymbol{\mu}^T) \text{diag}(\boldsymbol{\sigma})^{-1} \quad (7.82)$$

where $\boldsymbol{\mu} = \bar{\mathbf{x}}$ is the empirical mean, and $\boldsymbol{\sigma}$ is a vector of the empirical standard deviations.

7.2.4.3 Sum of squares and scatter matrix

The **sum of squares matrix** is $D \times D$ matrix defined by

$$\mathbf{S}_0 \triangleq \mathbf{X}^T \mathbf{X} = \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T = \sum_{n=1}^N \begin{pmatrix} x_{n,1}^2 & \cdots & x_{n,1} x_{n,D} \\ & \ddots & \\ x_{n,D} x_{n,1} & \cdots & x_{n,D}^2 \end{pmatrix} \quad (7.83)$$

The **scatter matrix** is a $D \times D$ matrix defined by

$$\mathbf{S}_{\bar{\mathbf{x}}} \triangleq \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T = \left(\sum_n \mathbf{x}_n \mathbf{x}_n^T \right) - N \bar{\mathbf{x}} \bar{\mathbf{x}}^T \quad (7.84)$$

We see that this is the sum of squares matrix applied to the mean-centered data. More precisely, define $\tilde{\mathbf{X}}$ to be a version of $\mathbf{X}$ where we subtract the mean $\bar{\mathbf{x}} = \frac{1}{N} \mathbf{X}^T \mathbf{1}_N$ off every row. Hence we can compute the centered data matrix using

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \bar{\mathbf{x}}^T = \mathbf{X} - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^T \mathbf{X} = \mathbf{C}_N \mathbf{X} \quad (7.85)$$

where

$$\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N} \mathbf{J}_N \quad (7.86)$$

is the **centering matrix**, and $\mathbf{J}_N = \mathbf{1}_N \mathbf{1}_N^\top$ is a matrix of all 1s. The scatter matrix can now be computed as follows:

$$\mathbf{S}_{\bar{\mathbf{x}}} = \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \mathbf{X}^\top \mathbf{C}_N^\top \mathbf{C}_N \mathbf{X} = \mathbf{X}^\top \mathbf{C}_N \mathbf{X} \quad (7.87)$$

where we exploited the fact that $\mathbf{C}_N$ is symmetric and idempotent, i.e., $\mathbf{C}_N^k = \mathbf{C}_N$ for $k = 1, 2, \dots$ (since once we subtract the mean, subtracting it again has no effect).

7.2.4.4 Gram matrix

The $N \times N$ matrix $\mathbf{X}\mathbf{X}^\top$ is a matrix of inner products called the **Gram matrix**:

$$\mathbf{K} \triangleq \mathbf{X}\mathbf{X}^\top = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \cdots & \mathbf{x}_1^\top \mathbf{x}_N \\ & \ddots & \\ \mathbf{x}_N^\top \mathbf{x}_1 & \cdots & \mathbf{x}_N^\top \mathbf{x}_N \end{pmatrix} \quad (7.88)$$

Sometimes we want to compute the inner products of the mean-centered data vectors, $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top$. However, if we are working with a feature similarity matrix instead of raw features, we will only have access to $\mathbf{K}$, not $\mathbf{X}$. (We will see examples of this in Section 20.4.4 and Section 20.4.6.) Fortunately, we can compute $\tilde{\mathbf{K}}$ from $\mathbf{K}$ using the **double centering trick**:

$$\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top = \mathbf{C}_N \mathbf{K} \mathbf{C}_N = \mathbf{K} - \frac{1}{N} \mathbf{J} \mathbf{K} - \frac{1}{N} \mathbf{K} \mathbf{J} + \frac{1}{N^2} \mathbf{1}^\top \mathbf{K} \mathbf{1} \quad (7.89)$$

This subtracts the row means and column means from $\mathbf{K}$, and adds back the global mean that gets subtracted twice, so that both row means and column means of $\tilde{\mathbf{K}}$ are equal to zero.

To see why Equation (7.89) is true, consider the scalar form:

$$\tilde{K}_{ij} = \tilde{\mathbf{x}}_i^\top \tilde{\mathbf{x}}_j = (\mathbf{x}_i - \frac{1}{N} \sum_{k=1}^N \mathbf{x}_k)(\mathbf{x}_j - \frac{1}{N} \sum_{l=1}^N \mathbf{x}_l) \quad (7.90)$$

$$= \mathbf{x}_i^\top \mathbf{x}_j - \frac{1}{N} \sum_{k=1}^N \mathbf{x}_i^\top \mathbf{x}_k - \frac{1}{N} \sum_{k=1}^N \mathbf{x}_j^\top \mathbf{x}_k + \frac{1}{N^2} \sum_{k=1}^N \sum_{l=1}^N \mathbf{x}_k^\top \mathbf{x}_l \quad (7.91)$$

7.2.4.5 Distance matrix

Let $\mathbf{X}$ be $N_x \times D$ datamatrix, and $\mathbf{Y}$ be another $N_y \times D$ datamatrix. We can compute the squared pairwise distances between these using

$$\mathbf{D}_{ij} = (\mathbf{x}_i - \mathbf{y}_j)^\top (\mathbf{x}_i - \mathbf{y}_j) = \|\mathbf{x}_i\|^2 - 2\mathbf{x}_i^\top \mathbf{y}_j + \|\mathbf{y}_j\|^2 \quad (7.92)$$

Let us now write this in matrix form. Let $\hat{\mathbf{x}} = [\|\mathbf{x}_1\|^2; \dots; \|\mathbf{x}_{N_x}\|^2] = \text{diag}(\mathbf{X}\mathbf{X}^\top)$ be a vector where each element is the squared norm of the examples in $\mathbf{X}$, and define $\hat{\mathbf{y}}$ similarly. Then we have

$$\mathbf{D} = \hat{\mathbf{x}} \mathbf{1}_{N_y}^\top - 2\mathbf{X}\mathbf{Y}^\top + \mathbf{1}_{N_x} \hat{\mathbf{y}}^\top \quad (7.93)$$

In the case that $\mathbf{X} = \mathbf{Y}$, we have

$$\mathbf{D} = \hat{\mathbf{x}} \mathbf{1}_N^\top - 2\mathbf{X}\mathbf{X}^\top + \mathbf{1}_N \hat{\mathbf{x}}^\top \quad (7.94)$$

This vectorized computation is often much faster than using for loops.

7.2.5 Kronecker products *

If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is a $p \times q$ matrix, then the **Kronecker product** $\mathbf{A} \otimes \mathbf{B}$ is the $mp \times nq$ block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \quad (7.95)$$

For example,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \otimes \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{11}b_{12} & a_{11}b_{13} & a_{12}b_{11} & a_{12}b_{12} & a_{12}b_{13} \\ a_{11}b_{21} & a_{11}b_{22} & a_{11}b_{23} & a_{12}b_{21} & a_{12}b_{22} & a_{12}b_{23} \\ a_{21}b_{11} & a_{21}b_{12} & a_{21}b_{13} & a_{22}b_{11} & a_{22}b_{12} & a_{22}b_{13} \\ a_{21}b_{21} & a_{21}b_{22} & a_{21}b_{23} & a_{22}b_{21} & a_{22}b_{22} & a_{22}b_{23} \\ a_{31}b_{11} & a_{31}b_{12} & a_{31}b_{13} & a_{32}b_{11} & a_{32}b_{12} & a_{32}b_{13} \\ a_{31}b_{21} & a_{31}b_{22} & a_{31}b_{23} & a_{32}b_{21} & a_{32}b_{22} & a_{32}b_{23} \end{bmatrix} \quad (7.96)$$

Here are some useful identities:

$$(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1} \quad (7.97)$$

$$(\mathbf{A} \otimes \mathbf{B})\text{vec}(\mathbf{C}) = \text{vec}(\mathbf{BCA}^T) \quad (7.98)$$

where $\text{vec}(\mathbf{M})$ stacks the *columns* of $\mathbf{M}$. (If we stack along the rows, we get $(\mathbf{A} \otimes \mathbf{B})\text{vec}(\mathbf{C}) = \text{vec}(\mathbf{ACB}^T)$.) See [Loa00] for a list of other useful properties.

7.2.6 Einstein summation *

Einstein summation, or **einsum** for short, is a notational shortcut for working with tensors. The convention was introduced by Einstein [Ein16, sec 5], who later joked to a friend, “I have made a great discovery in mathematics; I have suppressed the summation sign every time that the summation must be made over an index which occurs twice...” [Pai05, p.216]. For example, instead of writing matrix multiplication as $C_{ij} = \sum_k A_{ik}B_{kj}$, we can just write it as $C_{ij} = A_{ik}B_{kj}$, where we drop the $\sum_k$.

As a more complex example, suppose we have a 3d tensor S_{ntk} where n indexes examples in the batch, t indexes locations in the sequence, and k indexes words in a one-hot representation. Let W_{kd} be an embedding matrix that maps sparse one-hot vectors $\mathbb{R}^k$ to dense vectors in $\mathbb{R}^d$. We can convert the batch of sequences of one-hots to a batch of sequences of embeddings as follows:

$$E_{ntd} = \sum_k S_{ntk}W_{kd} \quad (7.99)$$

We can compute the sum of the embedding vectors for each sequence (to get a global representation of each bag of words) as follows:

$$E_{nd} = \sum_k \sum_t S_{ntk}W_{kd} \quad (7.100)$$

Finally we can pass each sequence's vector representation through another linear transform V_{dc} to map to the logits over a classifier with c labels:

$$L_{nc} = \sum_d E_{nd} V_{dc} = \sum_d \sum_k \sum_t S_{ntk} W_{kd} V_{dc} \quad (7.101)$$

In einsum notation, we write $L_{nc} = S_{ntk} W_{kd} V_{dc}$. We sum over k and d because those indices occur twice on the RHS. We sum over t because that index does not occur on the LHS.

Einsum is implemented in NumPy, Tensorflow, PyTorch, etc. What makes it particularly useful is that it can perform the relevant tensor multiplications in complex expressions in an optimal order, so as to minimize time and intermediate memory allocation.² The library is best illustrated by the examples in [einsum_demo.ipynb](#).

Note that the speed of einsum depends on the order in which the operations are performed, which depends on the shapes of the relevant arguments. The optimal ordering minimizes the treewidth of the resulting computation graph, as explained in [GASG18]. In general, the time to compute the optimal ordering is exponential in the number of arguments, so it is common to use a greedy approximation. However, if we expect to repeat the same calculation many times, using tensors of the same shape but potentially different content, we can compute the optimal ordering once and reuse it multiple times.

7.3 Matrix inversion

In this section, we discuss how to invert different kinds of matrices.

7.3.1 The inverse of a square matrix

The **inverse** of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is denoted $\mathbf{A}^{-1}$, and is the unique matrix such that

$$\mathbf{A}^{-1} \mathbf{A} = \mathbf{I} = \mathbf{A} \mathbf{A}^{-1}. \quad (7.102)$$

Note that $\mathbf{A}^{-1}$ exists if and only if $\det(\mathbf{A}) \neq 0$. If $\det(\mathbf{A}) = 0$, it is called a **singular** matrix.

The following are properties of the inverse; all assume that $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ are non-singular:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \quad (7.103)$$

$$(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \quad (7.104)$$

$$(\mathbf{A}^{-1})^\top = (\mathbf{A}^\top)^{-1} \triangleq \mathbf{A}^{-T} \quad (7.105)$$

For the case of a 2×2 matrix, the expression for $\mathbf{A}^{-1}$ is simple enough to give explicitly. We have

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad (7.106)$$

For a block diagonal matrix, the inverse is obtained by simply inverting each block separately, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{pmatrix} \quad (7.107)$$

2. These optimizations are implemented in the **opt-einsum** library [GASG18]. Its core functionality is included in NumPy and JAX einsum functions, provided you set `optimize=True` parameter.

7.3.2 Schur complements *

In this section, we review some useful results concerning block structured matrices.

Theorem 7.3.1 (Inverse of a partitioned matrix). *Consider a general partitioned matrix*

$$\mathbf{M} = \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \quad (7.108)$$

where we assume $\mathbf{E}$ and $\mathbf{H}$ are invertible. We have

$$\mathbf{M}^{-1} = \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \quad (7.109)$$

$$= \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \quad (7.110)$$

where

$$\mathbf{M}/\mathbf{H} \triangleq \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} \quad (7.111)$$

$$\mathbf{M}/\mathbf{E} \triangleq \mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F} \quad (7.112)$$

We say that $\mathbf{M}/\mathbf{H}$ is the **Schur complement** of $\mathbf{M}$ wrt $\mathbf{H}$, and $\mathbf{M}/\mathbf{E}$ is the Schur complement of $\mathbf{M}$ wrt $\mathbf{E}$.

Equation (7.109) and Equation (7.110) are called the **partitioned inverse formulae**.

Proof. If we could block diagonalize $\mathbf{M}$, it would be easier to invert. To zero out the top right block of $\mathbf{M}$ we can pre-multiply as follows

$$\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \quad (7.113)$$

Similarly, to zero out the bottom left we can post-multiply as follows

$$\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix} \quad (7.114)$$

Putting it all together we get

$$\underbrace{\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}}_{\mathbf{M}} \underbrace{\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix}}_{\mathbf{Z}} = \underbrace{\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix}}_{\mathbf{W}} \quad (7.115)$$

Taking the inverse of both sides yields

$$\mathbf{Z}^{-1}\mathbf{M}^{-1}\mathbf{X}^{-1} = \mathbf{W}^{-1} \quad (7.116)$$

$$\mathbf{M}^{-1} = \mathbf{Z}\mathbf{W}^{-1}\mathbf{X} \quad (7.117)$$

Substituting in the definitions we get

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \quad (7.118)$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \quad (7.119)$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \quad (7.120)$$

Alternatively, we could have decomposed the matrix $\mathbf{M}$ in terms of $\mathbf{E}$ and $\mathbf{M}/\mathbf{E} = (\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})$, yielding

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \quad (7.121)$$

□

7.3.3 The matrix inversion lemma *

Equating the top left block of the first matrix in Equation (7.119) with the top left block of the matrix in Equation (7.121)

$$(\mathbf{M}/\mathbf{H})^{-1} = (\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G})^{-1} = \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})^{-1}\mathbf{G}\mathbf{E}^{-1} \quad (7.122)$$

This is known as the **matrix inversion lemma** or the **Sherman-Morrison-Woodbury formula**.

A typical application in machine learning is the following. Let $\mathbf{X}$ be an $N \times D$ data matrix, and $\mathbf{\Sigma}$ be $N \times N$ diagonal matrix. Then we have (using the substitutions $\mathbf{E} = \mathbf{\Sigma}$, $\mathbf{F} = \mathbf{G}^T = \mathbf{X}$, and $\mathbf{H}^{-1} = -\mathbf{I}$) the following result:

$$(\mathbf{\Sigma} + \mathbf{X}\mathbf{X}^T)^{-1} = \mathbf{\Sigma}^{-1} - \mathbf{\Sigma}^{-1}\mathbf{X}(\mathbf{I} + \mathbf{X}^T\mathbf{\Sigma}^{-1}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{\Sigma}^{-1} \quad (7.123)$$

The LHS takes $O(N^3)$ time to compute, the RHS takes time $O(D^3)$ to compute.

Another application concerns computing a **rank one update** of an inverse matrix. Let $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = \mathbf{u}$, $\mathbf{G} = \mathbf{v}^T$, and $\mathbf{H} = -1$. Then we have

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^T)^{-1} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{u}(-1 - \mathbf{v}^T\mathbf{A}^{-1}\mathbf{u})^{-1}\mathbf{v}^T\mathbf{A}^{-1} \quad (7.124)$$

$$= \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^T\mathbf{A}^{-1}}{1 + \mathbf{v}^T\mathbf{A}^{-1}\mathbf{u}} \quad (7.125)$$

This is known as the **Sherman-Morrison formula**.

7.3.4 Matrix determinant lemma *

We now use the above results to derive an efficient way to compute the determinant of a block-structured matrix.

From Equation (7.115), we have

$$|\mathbf{X}||\mathbf{M}||\mathbf{Z}| = |\mathbf{W}| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \quad (7.126)$$

$$\left| \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \right| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \quad (7.127)$$

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| \quad (7.128)$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}|}{|\mathbf{H}|} \quad (7.129)$$

So we can see that $\mathbf{M}/\mathbf{H}$ acts somewhat like a division operator (hence the notation).

Furthermore, we have

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| = |\mathbf{M}/\mathbf{E}||\mathbf{E}| \quad (7.130)$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}/\mathbf{E}||\mathbf{E}|}{|\mathbf{H}|} \quad (7.131)$$

$$|\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}| = |\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F}||\mathbf{H}^{-1}||\mathbf{E}| \quad (7.132)$$

Hence (setting $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = -\mathbf{u}$, $\mathbf{G} = \mathbf{v}^\top$, $\mathbf{H} = 1$) we have

$$|\mathbf{A} + \mathbf{u}\mathbf{v}^\top| = (1 + \mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u})|\mathbf{A}| \quad (7.133)$$

This is known as the **matrix determinant lemma**.

7.3.5 Application: deriving the conditionals of an MVN *

Consider a joint Gaussian of the form $p(\mathbf{x}_1, \mathbf{x}_2) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \quad (7.134)$$

In Section 3.2.3, we claimed that

$$p(\mathbf{x}_1|\mathbf{x}_2) = \mathcal{N}(\mathbf{x}_1|\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2), \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}) \quad (7.135)$$

In this section, we derive this result using Schur complements.

Let us factor the joint $p(\mathbf{x}_1, \mathbf{x}_2)$ as $p(\mathbf{x}_2)p(\mathbf{x}_1|\mathbf{x}_2)$ as follows:

$$p(\mathbf{x}_1, \mathbf{x}_2) \propto \exp \left\{ -\frac{1}{2} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^\top \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix}^{-1} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix} \right\} \quad (7.136)$$

Using Equation (7.118) the above exponent becomes

$$p(\mathbf{x}_1, \mathbf{x}_2) \propto \exp \left\{ -\frac{1}{2} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^\top \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22})^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{22}^{-1} \end{pmatrix} \right. \quad (7.137)$$

$$\left. \times \begin{pmatrix} \mathbf{I} & -\boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix} \right\} \quad (7.138)$$

$$= \exp \left\{ -\frac{1}{2} (\mathbf{x}_1 - \boldsymbol{\mu}_1 - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2))^\top (\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22})^{-1} \right. \quad (7.139)$$

$$\left. (\mathbf{x}_1 - \boldsymbol{\mu}_1 - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2)) \right\} \times \exp \left\{ -\frac{1}{2} (\mathbf{x}_2 - \boldsymbol{\mu}_2)^\top \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \right\} \quad (7.140)$$

This is of the form

$$\exp(\text{quadratic form in } \mathbf{x}_1, \mathbf{x}_2) \times \exp(\text{quadratic form in } \mathbf{x}_2) \quad (7.141)$$

Hence we have successfully factorized the joint as

$$p(\mathbf{x}_1, \mathbf{x}_2) = p(\mathbf{x}_1 | \mathbf{x}_2) p(\mathbf{x}_2) \quad (7.142)$$

$$= \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2}) \mathcal{N}(\mathbf{x}_2 | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \quad (7.143)$$

where the parameters of the conditional distribution can be read off from the above equations using

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \quad (7.144)$$

$$\boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21} \quad (7.145)$$

We can also use the fact that $|\mathbf{M}| = |\mathbf{M}/\mathbf{H}| |\mathbf{H}|$ to check the normalization constants are correct:

$$(2\pi)^{(d_1+d_2)/2} |\boldsymbol{\Sigma}|^{\frac{1}{2}} = (2\pi)^{(d_1+d_2)/2} (|\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}| |\boldsymbol{\Sigma}_{22}|)^{\frac{1}{2}} \quad (7.146)$$

$$= (2\pi)^{d_1/2} |\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}} (2\pi)^{d_2/2} |\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}} \quad (7.147)$$

where $d_1 = \dim(\mathbf{x}_1)$ and $d_2 = \dim(\mathbf{x}_2)$.

7.4 Eigenvalue decomposition (EVD)

In this section, we review some standard material on the **eigenvalue decomposition** or **EVD** of square (real-valued) matrices.

7.4.1 Basics

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, we say that $\lambda \in \mathbb{R}$ is an **eigenvalue** of $\mathbf{A}$ and $\mathbf{u} \in \mathbb{R}^n$ is the corresponding **eigenvector** if

$$\mathbf{A}\mathbf{u} = \lambda\mathbf{u}, \quad \mathbf{u} \neq \mathbf{0} . \quad (7.148)$$

Intuitively, this definition means that multiplying $\mathbf{A}$ by the vector $\mathbf{u}$ results in a new vector that points in the same direction as $\mathbf{u}$, but is scaled by a factor λ . For example, if $\mathbf{A}$ is a rotation matrix, then $\mathbf{u}$ is the axis of rotation and $\lambda = 1$.

Note that for any eigenvector $\mathbf{u} \in \mathbb{R}^n$, and scalar $c \in \mathbb{R}$,

$$\mathbf{A}(c\mathbf{u}) = c\mathbf{A}\mathbf{u} = c\lambda\mathbf{u} = \lambda(c\mathbf{u}) \quad (7.149)$$

Hence $c\mathbf{u}$ is also an eigenvector. For this reason when we talk about “the” eigenvector associated with λ , we usually assume that the eigenvector is normalized to have length 1 (this still creates some ambiguity, since $\mathbf{u}$ and $-\mathbf{u}$ will both be eigenvectors, but we will have to live with this).

We can rewrite the equation above to state that $(\lambda, \mathbf{x})$ is an eigenvalue-eigenvector pair of $\mathbf{A}$ if

$$(\lambda\mathbf{I} - \mathbf{A})\mathbf{u} = \mathbf{0}, \quad \mathbf{u} \neq \mathbf{0} . \quad (7.150)$$

Now $(\lambda\mathbf{I} - \mathbf{A})\mathbf{u} = \mathbf{0}$ has a non-zero solution to $\mathbf{u}$ if and only if $(\lambda\mathbf{I} - \mathbf{A})$ has a non-empty nullspace, which is only the case if $(\lambda\mathbf{I} - \mathbf{A})$ is singular, i.e.,

$$\det(\lambda\mathbf{I} - \mathbf{A}) = 0 . \quad (7.151)$$

This is called the **characteristic equation** of $\mathbf{A}$. (See Exercise 7.2.) The n solutions of this equation are the n (possibly complex-valued) eigenvalues λ_i , and $\mathbf{u}_i$ are the corresponding eigenvectors. It is standard to sort the eigenvectors in order of their eigenvalues, with the largest magnitude ones first.

The following are properties of eigenvalues and eigenvectors.

- The trace of a matrix is equal to the sum of its eigenvalues,

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i . \quad (7.152)$$

- The determinant of $\mathbf{A}$ is equal to the product of its eigenvalues,

$$\det(\mathbf{A}) = \prod_{i=1}^n \lambda_i . \quad (7.153)$$

- The rank of $\mathbf{A}$ is equal to the number of non-zero eigenvalues of $\mathbf{A}$.
- If $\mathbf{A}$ is non-singular then $1/\lambda_i$ is an eigenvalue of $\mathbf{A}^{-1}$ with associated eigenvector $\mathbf{u}_i$, i.e., $\mathbf{A}^{-1}\mathbf{u}_i = (1/\lambda_i)\mathbf{u}_i$.
- The eigenvalues of a diagonal or triangular matrix are just the diagonal entries.

7.4.2 Diagonalization

We can write all the eigenvector equations simultaneously as

$$\mathbf{A}\mathbf{U} = \mathbf{U}\mathbf{\Lambda} \quad (7.154)$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ are the eigenvectors of $\mathbf{A}$ and $\mathbf{\Lambda}$ is a diagonal matrix whose entries are the eigenvalues of $\mathbf{A}$, i.e.,

$$\mathbf{U} \in \mathbb{R}^{n \times n} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \\ | & | & & | \end{bmatrix}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n) . \quad (7.155)$$

If the eigenvectors of $\mathbf{A}$ are linearly independent, then the matrix $\mathbf{U}$ will be invertible, so

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}. \quad (7.156)$$

A matrix that can be written in this form is called **diagonalizable**.

7.4.3 Eigenvalues and eigenvectors of symmetric matrices

When $\mathbf{A}$ is real and symmetric, it can be shown that all the eigenvalues are real, and the eigenvectors are **orthonormal**, i.e., $\mathbf{u}_i^\top \mathbf{u}_j = 0$ if $i \neq j$, and $\mathbf{u}_i^\top \mathbf{u}_i = 1$, where $\mathbf{u}_i$ are the eigenvectors. In matrix form, this becomes $\mathbf{U}^\top \mathbf{U} = \mathbf{U} \mathbf{U}^\top = \mathbf{I}$; hence we see that $\mathbf{U}$ is an orthogonal matrix.

We can therefore represent $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top = \begin{pmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_1^\top & - \\ - & \mathbf{u}_2^\top & - \\ & \vdots & \\ - & \mathbf{u}_n^\top & - \end{pmatrix} \quad (7.157)$$

$$= \lambda_1 \begin{pmatrix} | \\ \mathbf{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_1^\top & - \end{pmatrix} + \cdots + \lambda_n \begin{pmatrix} | \\ \mathbf{u}_n \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_n^\top & - \end{pmatrix} = \sum_{i=1}^n \lambda_i \mathbf{u}_i \mathbf{u}_i^\top \quad (7.158)$$

Thus multiplying by any symmetric matrix $\mathbf{A}$ can be interpreted as multiplying by a rotation matrix $\mathbf{U}^\top$, a scaling matrix $\mathbf{\Lambda}$, followed by an inverse rotation $\mathbf{U}$.

Once we have diagonalized a matrix, it is easy to invert. Since $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, where $\mathbf{U}^\top = \mathbf{U}^{-1}$, we have

$$\mathbf{A}^{-1} = \mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^\top = \sum_{i=1}^d \frac{1}{\lambda_i} \mathbf{u}_i \mathbf{u}_i^\top \quad (7.159)$$

This corresponds to rotating, unscaling, and then rotating back.

7.4.3.1 Checking for positive definiteness

We can also use the diagonalization property to show that a symmetric matrix is positive definite iff all its eigenvalues are positive. To see this, note that

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{x} = \mathbf{y}^\top \mathbf{\Lambda} \mathbf{y} = \sum_{i=1}^n \lambda_i y_i^2 \quad (7.160)$$

where $\mathbf{y} = \mathbf{U}^\top \mathbf{x}$. Because y_i^2 is always nonnegative, the sign of this expression depends entirely on the λ_i 's. If all $\lambda_i > 0$, then the matrix is positive definite; if all $\lambda_i \geq 0$, it is positive semidefinite. Likewise, if all $\lambda_i < 0$ or $\lambda_i \leq 0$, then $\mathbf{A}$ is negative definite or negative semidefinite respectively. Finally, if $\mathbf{A}$ has both positive and negative eigenvalues, it is indefinite.

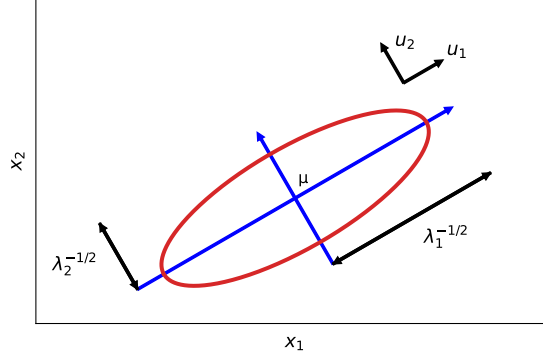


Figure 7.6: Visualization of a level set of the quadratic form $(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{A}(\mathbf{x} - \boldsymbol{\mu})$ in 2d. The major and minor axes of the ellipse are defined by the first two eigenvectors of $\mathbf{A}$, namely $\mathbf{u}_1$ and $\mathbf{u}_2$. Adapted from Figure 2.7 of [Bis06]. Generated by [gaussEvec.ipynb](#).

7.4.4 Geometry of quadratic forms

A **quadratic form** is a function that can be written as

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} \quad (7.161)$$

where $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{A}$ is a positive definite, symmetric n -by- n matrix. Let $\mathbf{A} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top$ be a diagonalization of $\mathbf{A}$ (see Section 7.4.3). Hence we can write

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top \mathbf{x} = \mathbf{y}^\top \boldsymbol{\Lambda} \mathbf{y} = \sum_{i=1}^n \lambda_i y_i^2 \quad (7.162)$$

where $y_i = \mathbf{x}^\top \mathbf{u}_i$ and $\lambda_i > 0$ (since $\mathbf{A}$ is positive definite). The level sets of $f(\mathbf{x})$ define hyper-ellipsoids. For example, in 2d, we have

$$\lambda_1 y_1^2 + \lambda_2 y_2^2 = r \quad (7.163)$$

which is the equation of a 2d ellipse. This is illustrated in Figure 7.6. The eigenvectors determine the orientation of the ellipse, and the eigenvalues determine how elongated it is. In particular, the major and minor semi-axes of the ellipse satisfy $a^{-2} = \lambda_1$ and $b^{-2} = \lambda_2$. In the case of a Gaussian distribution, we have $\mathbf{A} = \boldsymbol{\Sigma}^{-1}$, so small values of λ_i correspond to directions where the posterior has low precision and hence high variance.

7.4.5 Standardizing and whitening data

Suppose we have a dataset $\mathbf{X} \in \mathbb{R}^{N \times D}$. It is common to preprocess the data so that each column has zero mean and unit variance. This is called standardizing the data, as we discuss in Section 10.2.8. Although standardizing forces the variance to be 1, it does not remove correlation between the columns. To do that, we must **whiten** the data. To define this, let the empirical covariance matrix

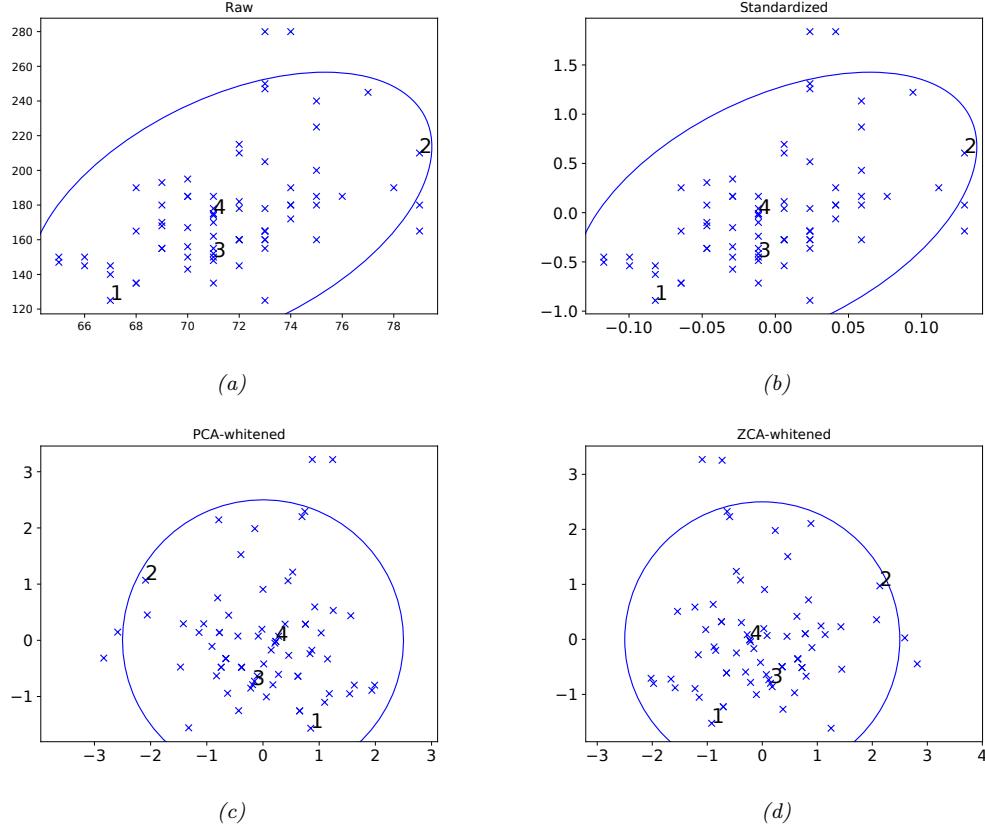


Figure 7.7: (a) Height/weight data. (b) Standardized. (c) PCA Whitening. (d) ZCA whitening. Numbers refer to the first 4 datapoints, but there are 73 datapoints in total. Generated by [height_weight_whiten_plot.ipynb](#).

be $\Sigma = \frac{1}{N} \mathbf{X}^T \mathbf{X}$, and let $\Sigma = \mathbf{E} \mathbf{D} \mathbf{E}^T$ be its diagonalization. Equivalently, let $[\mathbf{U}, \mathbf{S}, \mathbf{V}]$ be the SVD of $\frac{1}{\sqrt{N}} \mathbf{X}$ (so $\mathbf{E} = \mathbf{V}$ and $\mathbf{D} = \mathbf{S}^2$, as we discuss in Section 20.1.3.3.) Now define

$$\mathbf{W}_{pca} = \mathbf{D}^{-\frac{1}{2}} \mathbf{E}^T \quad (7.164)$$

This is called the **PCA whitening** matrix. (We discuss PCA in Section 20.1.) Let $\mathbf{y} = \mathbf{W}_{pca} \mathbf{x}$ be a transformed vector. We can check that its covariance is white as follows:

$$\text{Cov}[\mathbf{y}] = \mathbf{W} \mathbb{E}[\mathbf{x} \mathbf{x}^T] \mathbf{W}^T = \mathbf{W} \Sigma \mathbf{W}^T = (\mathbf{D}^{-\frac{1}{2}} \mathbf{E}^T)(\mathbf{E} \mathbf{D} \mathbf{E}^T)(\mathbf{E} \mathbf{D}^{-\frac{1}{2}}) = \mathbf{I} \quad (7.165)$$

The whitening matrix is not unique, since any rotation of it, $\mathbf{W} = \mathbf{R} \mathbf{W}_{pca}$, will still maintain the whitening property, i.e., $\mathbf{W}^T \mathbf{W} = \Sigma^{-1}$. For example, if we take $\mathbf{R} = \mathbf{E}$, we get

$$\mathbf{W}_{zca} = \mathbf{E} \mathbf{D}^{-\frac{1}{2}} \mathbf{E}^T = \Sigma^{-\frac{1}{2}} = \mathbf{V} \mathbf{S}^{-1} \mathbf{V}^T \quad (7.166)$$

This is called **Mahalanobis whitening** or **ZCA**. (ZCA stands for “zero-phase component analysis”, and was introduced in [BS97].) The advantage of ZCA whitening over PCA whitening is that the resulting transformed data is as close as possible to the original data (in the least squares sense) [Amo17]. This is illustrated in Figure 7.7. When applied to images, the ZCA transformed data vectors still look like images. This is useful when the method is used inside a deep learning system [KH09].

7.4.6 Power method

We now describe a simple iterative method for computing the eigenvector corresponding to the largest eigenvalue of a real, symmetric matrix; this is called the **power method**. This can be useful when the matrix is very large but sparse. For example, it is used by Google’s **PageRank** to compute the stationary distribution of the transition matrix of the world wide web (a matrix of size about 3 billion by 3 billion!). In Section 7.4.7, we will see how to use this method to compute subsequent eigenvectors and values.

Let $\mathbf{A}$ be a matrix with orthonormal eigenvectors $\mathbf{u}_i$ and eigenvalues $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_m| \geq 0$, so $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$. Let $\mathbf{v}_{(0)}$ be an arbitrary vector in the range of $\mathbf{A}$, so $\mathbf{A}\mathbf{x} = \mathbf{v}_{(0)}$ for some $\mathbf{x}$. Hence we can write $\mathbf{v}_{(0)}$ as

$$\mathbf{v}_0 = \mathbf{U}(\mathbf{\Lambda}\mathbf{U}^\top\mathbf{x}) = a_1\mathbf{u}_1 + \dots + a_m\mathbf{u}_m \quad (7.167)$$

for some constants a_i . We can now repeatedly multiply $\mathbf{v}$ by $\mathbf{A}$ and renormalize:

$$\mathbf{v}_t \propto \mathbf{A}\mathbf{v}_{t-1} \quad (7.168)$$

(We normalize at each iteration for numerical stability.)

Since $\mathbf{v}_t$ is a multiple of $\mathbf{A}^t\mathbf{v}_0$, we have

$$\mathbf{v}_t \propto a_1\lambda_1^t\mathbf{u}_1 + a_2\lambda_2^t\mathbf{u}_2 + \dots + a_m\lambda_m^t\mathbf{u}_m \quad (7.169)$$

$$= \lambda_1^t (a_1\mathbf{u}_1 + a_1(\lambda_2/\lambda_1)^t\mathbf{u}_2 + \dots + a_m(\lambda_m/\lambda_1)^t\mathbf{u}_m) \quad (7.170)$$

$$\rightarrow \lambda_1^t a_1\mathbf{u}_1 \quad (7.171)$$

since $|\lambda_k/\lambda_1| < 1$ for $k > 1$ (assuming the eigenvalues are sorted in descending order). So we see that this converges to $\mathbf{u}_1$, although not very quickly (the error is reduced by approximately $|\lambda_2/\lambda_1|$ at each iteration). The only requirement is that the initial guess satisfy $\mathbf{v}_0^\top\mathbf{u}_1 \neq 0$, which will be true for a random $\mathbf{v}_0$ with high probability.

We now discuss how to compute the corresponding eigenvalue, λ_1 . Define the **Rayleigh quotient** to be

$$R(\mathbf{A}, \mathbf{x}) \triangleq \frac{\mathbf{x}^\top \mathbf{A} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}} \quad (7.172)$$

Hence

$$R(\mathbf{A}, \mathbf{u}_i) = \frac{\mathbf{u}_i^\top \mathbf{A} \mathbf{u}_i}{\mathbf{u}_i^\top \mathbf{u}_i} = \frac{\lambda_i \mathbf{u}_i^\top \mathbf{u}_i}{\mathbf{u}_i^\top \mathbf{u}_i} = \lambda_i \quad (7.173)$$

Thus we can easily compute λ_1 from $\mathbf{u}_1$ and $\mathbf{A}$. See [power_method_demo.ipynb](#) for a demo.

7.4.7 Deflation

Suppose we have computed the first eigenvector and value $\mathbf{u}_1, \lambda_1$ by the power method. We now describe how to compute subsequent eigenvectors and values. Since the eigenvectors are orthonormal, and the eigenvalues are real, we can project out the $\mathbf{u}_1$ component from the matrix as follows:

$$\mathbf{A}^{(2)} = (\mathbf{I} - \mathbf{u}_1 \mathbf{u}_1^\top) \mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \mathbf{u}_1 \mathbf{u}_1^\top \mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \lambda_1 \mathbf{u}_1 \mathbf{u}_1^\top \quad (7.174)$$

This is called matrix **deflation**. We can then apply the power method to $\mathbf{A}^{(2)}$, which will find the largest eigenvector/value in the subspace orthogonal to $\mathbf{u}_1$.

In Section 20.1.2, we show that the optimal estimate $\hat{\mathbf{W}}$ for the PCA model (described in Section 20.1) is given by the first K eigenvectors of the empirical covariance matrix. Hence deflation can be used to implement PCA. It can also be modified to implement sparse PCA [Mac09].

7.4.8 Eigenvectors optimize quadratic forms

We can use matrix calculus to solve an optimization problem in a way that leads directly to eigenvalue/eigenvector analysis. Consider the following, equality constrained optimization problem:

$$\max_{\mathbf{x} \in \mathbb{R}^n} \mathbf{x}^\top \mathbf{A} \mathbf{x} \quad \text{subject to } \|\mathbf{x}\|_2^2 = 1 \quad (7.175)$$

for a symmetric matrix $\mathbf{A} \in \mathbb{S}^n$. A standard way of solving optimization problems with equality constraints is by forming the Lagrangian, an objective function that includes the equality constraints (see Section 8.5.1). The Lagrangian in this case can be given by

$$\mathcal{L}(\mathbf{x}, \lambda) = \mathbf{x}^\top \mathbf{A} \mathbf{x} + \lambda(1 - \mathbf{x}^\top \mathbf{x}) \quad (7.176)$$

where λ is called the Lagrange multiplier associated with the equality constraint. It can be established that for x^* to be a optimal point to the problem, the gradient of the Lagrangian has to be zero at x^* (this is not the only condition, but it is required). That is,

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda) = 2\mathbf{A}^\top \mathbf{x} - 2\lambda \mathbf{x} = \mathbf{0}. \quad (7.177)$$

Notice that this is just the linear equation $\mathbf{A} \mathbf{x} = \lambda \mathbf{x}$. This shows that the only points which can possibly maximize (or minimize) $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ assuming $\mathbf{x}^\top \mathbf{x} = 1$ are the eigenvectors of $\mathbf{A}$.

7.5 Singular value decomposition (SVD)

We now discuss the SVD, which generalizes EVD to rectangular matrices.

7.5.1 Basics

Any (real) $m \times n$ matrix $\mathbf{A}$ can be decomposed as

$$\mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^\top = \sigma_1 \begin{pmatrix} | \\ \mathbf{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_1^\top & - \end{pmatrix} + \cdots + \sigma_r \begin{pmatrix} | \\ \mathbf{u}_r \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_r^\top & - \end{pmatrix} \quad (7.178)$$

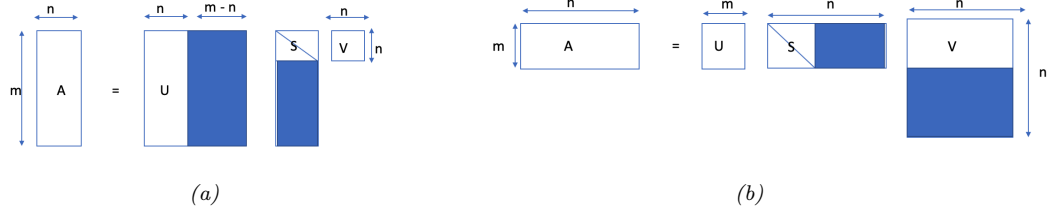


Figure 7.8: SVD decomposition of a matrix, $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$. The shaded parts of each matrix are not computed in the economy-sized version. (a) Tall skinny matrix. (b) Short wide matrix.

where $\mathbf{U}$ is an $m \times m$ whose columns are orthonormal (so $\mathbf{U}^T \mathbf{U} = \mathbf{I}_m$), $\mathbf{V}$ is $n \times n$ matrix whose rows and columns are orthonormal (so $\mathbf{V}^T \mathbf{V} = \mathbf{V}\mathbf{V}^T = \mathbf{I}_n$), and $\mathbf{S}$ is a $m \times n$ matrix containing the $r = \min(m, n)$ **singular values** $\sigma_i \geq 0$ on the main diagonal, with 0s filling the rest of the matrix. The columns of $\mathbf{U}$ are the left **singular vectors**, and the columns of $\mathbf{V}$ are the right singular vectors. This is called the **singular value decomposition** or **SVD** of the matrix. See Figure 7.8 for an example.

As is apparent from Figure 7.8a, if $m > n$, there are at most n singular values, so the last $m - n$ columns of $\mathbf{U}$ are irrelevant (since they will be multiplied by 0). The **economy sized SVD**, also called a **thin SVD**, avoids computing these unnecessary elements. In other words, if we write the $\mathbf{U}$ matrix as $\mathbf{U} = [\mathbf{U}_1, \mathbf{U}_2]$, we only compute $\mathbf{U}_1$. Figure 7.8b shows the opposite case, where $m < n$, where we represent $\mathbf{V} = [\mathbf{V}_1; \mathbf{V}_2]$, and only compute $\mathbf{V}_1$.

The cost of computing the SVD is $O(\min(mn^2, m^2n))$. Details on how it works can be found in standard linear algebra textbooks.

7.5.2 Connection between SVD and EVD

If $\mathbf{A}$ is real, symmetric and positive definite, then the singular values are equal to the eigenvalues, and the left and right singular vectors are equal to the eigenvectors (up to a sign change):

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T = \mathbf{U}\mathbf{S}\mathbf{U}^T = \mathbf{U}\mathbf{S}\mathbf{U}^{-1} \quad (7.179)$$

Note, however, that NumPy always returns the singular values in decreasing order, whereas the eigenvalues need not necessarily be sorted.

In general, for an arbitrary real matrix $\mathbf{A}$, if $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$, we have

$$\mathbf{A}^T \mathbf{A} = \mathbf{V}\mathbf{S}^T \mathbf{U}^T \mathbf{U}\mathbf{S}\mathbf{V}^T = \mathbf{V}(\mathbf{S}^T \mathbf{S})\mathbf{V}^T \quad (7.180)$$

Hence

$$(\mathbf{A}^T \mathbf{A})\mathbf{V} = \mathbf{V}\mathbf{D}_n \quad (7.181)$$

so the eigenvectors of $\mathbf{A}^T \mathbf{A}$ are equal to $\mathbf{V}$, the right singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}^T \mathbf{A}$ are equal to $\mathbf{D}_n = \mathbf{S}^T \mathbf{S}$, which is an $n \times n$ diagonal matrix containing the squared singular values. Similarly

$$\mathbf{A}\mathbf{A}^T = \mathbf{U}\mathbf{S}\mathbf{V}^T \mathbf{V}\mathbf{S}^T \mathbf{U}^T = \mathbf{U}(\mathbf{S}\mathbf{S}^T)\mathbf{U}^T \quad (7.182)$$

$$(\mathbf{A}\mathbf{A}^T)\mathbf{U} = \mathbf{U}\mathbf{D}_m \quad (7.183)$$

so the eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are equal to $\mathbf{U}$, the left singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}\mathbf{A}^\top$ are equal to $\mathbf{D}_m = \mathbf{S}\mathbf{S}^\top$, which is an $m \times m$ diagonal matrix containing the squared singular values. In summary,

$$\mathbf{U} = \text{evec}(\mathbf{A}\mathbf{A}^\top), \mathbf{V} = \text{evec}(\mathbf{A}^\top\mathbf{A}), \mathbf{D}_m = \text{eval}(\mathbf{A}\mathbf{A}^\top), \mathbf{D}_n = \text{eval}(\mathbf{A}^\top\mathbf{A}) \quad (7.184)$$

If we just use the computed (non-zero) parts in the economy-sized SVD, then we can define

$$\mathbf{D} = \mathbf{S}^2 = \mathbf{S}^\top\mathbf{S} = \mathbf{S}\mathbf{S}^\top \quad (7.185)$$

Note also that an EVD does not always exist, even for square $\mathbf{A}$, whereas an SVD always exists.

7.5.3 Pseudo inverse

The **Moore-Penrose pseudo-inverse** of $\mathbf{A}$, pseudo inverse denoted $\mathbf{A}^\dagger$, is defined as the unique matrix that satisfies the following 4 properties:

$$\mathbf{A}\mathbf{A}^\dagger\mathbf{A} = \mathbf{A}, \mathbf{A}^\dagger\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}^\dagger, (\mathbf{A}\mathbf{A}^\dagger)^\top = \mathbf{A}\mathbf{A}^\dagger, (\mathbf{A}^\dagger\mathbf{A})^\top = \mathbf{A}^\dagger\mathbf{A} \quad (7.186)$$

If $\mathbf{A}$ is square and non-singular, then $\mathbf{A}^\dagger = \mathbf{A}^{-1}$.

If $m > n$ (tall, skinny) and the columns of $\mathbf{A}$ are linearly independent (so $\mathbf{A}$ is full rank), then

$$\mathbf{A}^\dagger = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top \quad (7.187)$$

which is the same expression as arises in the normal equations (see Section 11.2.2.1). In this case, $\mathbf{A}^\dagger$ is a left inverse of $\mathbf{A}$ because

$$\mathbf{A}^\dagger\mathbf{A} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{A} = \mathbf{I} \quad (7.188)$$

but is not a right inverse because

$$\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top \quad (7.189)$$

only has rank n , and so cannot be the $m \times m$ identity matrix.

If $m < n$ (short, fat) and the rows of $\mathbf{A}$ are linearly independent (so $\mathbf{A}^\top$ is full rank), then the pseudo inverse is

$$\mathbf{A}^\dagger = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1} \quad (7.190)$$

In this case, $\mathbf{A}^\dagger$ is a right inverse of $\mathbf{A}$.

We can compute the pseudo inverse using the SVD decomposition $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. In particular, one can show that

$$\mathbf{A}^\dagger = \mathbf{V}[\text{diag}(1/\sigma_1, \dots, 1/\sigma_r, 0, \dots, 0)]\mathbf{U}^\top = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top \quad (7.191)$$

where r is the rank of the matrix, and where we define $\mathbf{S}^{-1} = \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1}, 0, \dots, 0)$. Indeed if the matrices were square and full rank we would have

$$(\mathbf{U}\mathbf{S}\mathbf{V}^\top)^{-1} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top \quad (7.192)$$

7.5.4 SVD and the range and null space of a matrix *

In this section, we show that the left and right singular vectors form an orthonormal basis for the range and null space.

From Equation (7.178) we have

$$\mathbf{A}\mathbf{x} = \sum_{j:\sigma_j>0} \sigma_j(\mathbf{v}_j^\top \mathbf{x})\mathbf{u}_j = \sum_{j=1}^r \sigma_j(\mathbf{v}_j^\top \mathbf{x})\mathbf{u}_j \quad (7.193)$$

where r is the rank of $\mathbf{A}$. Thus any $\mathbf{A}\mathbf{x}$ can be written as a linear combination of the left singular vectors $\mathbf{u}_1, \dots, \mathbf{u}_r$, so the range of $\mathbf{A}$ is given by

$$\text{range}(\mathbf{A}) = \text{span}(\{\mathbf{u}_j : \sigma_j > 0\}) \quad (7.194)$$

with dimension r .

To find a basis for the null space, let us now define a second vector $\mathbf{y} \in \mathbb{R}^n$ that is a linear combination solely of the right singular vectors for the zero singular values,

$$\mathbf{y} = \sum_{j:\sigma_j=0} c_j \mathbf{v}_j = \sum_{j=r+1}^n c_j \mathbf{v}_j \quad (7.195)$$

Since the $\mathbf{v}_j$'s are orthonormal, we have

$$\mathbf{A}\mathbf{y} = \mathbf{U} \begin{pmatrix} \sigma_1 \mathbf{v}_1^\top \mathbf{y} \\ \vdots \\ \sigma_r \mathbf{v}_r^\top \mathbf{y} \\ \sigma_{r+1} \mathbf{v}_{r+1}^\top \mathbf{y} \\ \vdots \\ \sigma_n \mathbf{v}_n^\top \mathbf{y} \end{pmatrix} = \mathbf{U} \begin{pmatrix} \sigma_1 0 \\ \vdots \\ \sigma_r 0 \\ 0 \mathbf{v}_{r+1}^\top \mathbf{y} \\ \vdots \\ 0 \mathbf{v}_n^\top \mathbf{y} \end{pmatrix} = \mathbf{U}\mathbf{0} = \mathbf{0} \quad (7.196)$$

Hence the right singular vectors form an orthonormal basis for the null space:

$$\text{nullspace}(\mathbf{A}) = \text{span}(\{\mathbf{v}_j : \sigma_j = 0\}) \quad (7.197)$$

with dimension $n - r$. We see that

$$\dim(\text{range}(\mathbf{A})) + \dim(\text{nullspace}(\mathbf{A})) = r + (n - r) = n \quad (7.198)$$

In words, this is often written as

$$\text{rank} + \text{nullity} = n \quad (7.199)$$

This is called the **rank-nullity theorem**. It follows from this that the rank of a matrix is the number of nonzero singular values.

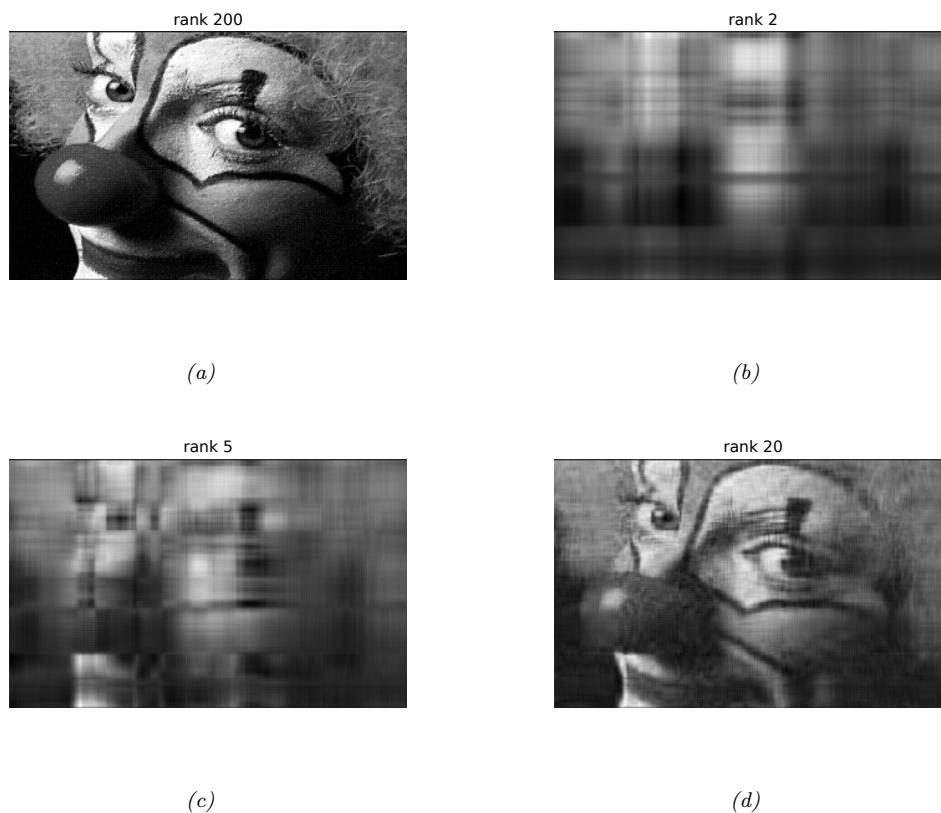


Figure 7.9: Low rank approximations to an image. Top left: The original image is of size 200×320 , so has rank 200. Subsequent images have ranks 2, 5, and 20. Generated by [svd_image_demo.ipynb](#).

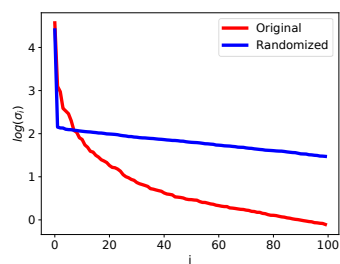


Figure 7.10: First 100 log singular values for the clown image (red line), and for a data matrix obtained by randomly shuffling the pixels (blue line). Generated by [svd_image_demo.ipynb](#). Adapted from Figure 14.24 of [HTF09].

7.5.5 Truncated SVD

Let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the SVD of $\mathbf{A}$, and let $\hat{\mathbf{A}}_K = \mathbf{U}_K\mathbf{S}_K\mathbf{V}_K^T$, where we use the first K columns of $\mathbf{U}$ and $\mathbf{V}$. This can be shown to be the optimal rank K approximation, in the sense that it minimizes $\|\mathbf{A} - \hat{\mathbf{A}}_K\|_F^2$.

If $K = r = \text{rank}(\mathbf{A})$, there is no error introduced by this decomposition. But if $K < r$, we incur some error. This is called a **truncated SVD**. If the singular values die off quickly, as is typical in natural data (see e.g., Figure 7.10), the error will be small. The total number of parameters needed to represent an $N \times D$ matrix using a rank K approximation is

$$NK + KD + K = K(N + D + 1) \quad (7.200)$$

As an example, consider the 200×320 pixel image in Figure 7.9(top left). This has 64,000 numbers in it. We see that a rank 20 approximation, with only $(200 + 320 + 1) \times 20 = 10,420$ numbers is a very good approximation.

One can show that the error in this rank- K approximation is given by

$$\|\mathbf{A} - \hat{\mathbf{A}}\|_F = \sum_{k=K+1}^r \sigma_k \quad (7.201)$$

where σ_k is the k 'th singular value of $\mathbf{A}$.

7.6 Other matrix decompositions *

In this section, we briefly review some other useful matrix decompositions.

7.6.1 LU factorization

We can factorize any square matrix $\mathbf{A}$ into a product of a lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$. For example,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}. \quad (7.202)$$

In general we may need to permute the entries in the matrix before creating this decomposition. To see this, suppose $a_{11} = 0$. Since $a_{11} = l_{11}u_{11}$, this means either l_{11} or u_{11} or both must be zero, but that would imply $\mathbf{L}$ or $\mathbf{U}$ are singular. To avoid this, the first step of the algorithm can simply reorder the rows so that the first element is nonzero. This is repeated for subsequent steps. We can denote this process by

$$\mathbf{PA} = \mathbf{LU} \quad (7.203)$$

where $\mathbf{P}$ is a permutation matrix, i.e., a square binary matrix where $P_{ij} = 1$ if row j gets permuted to row i . This is called **partial pivoting**.

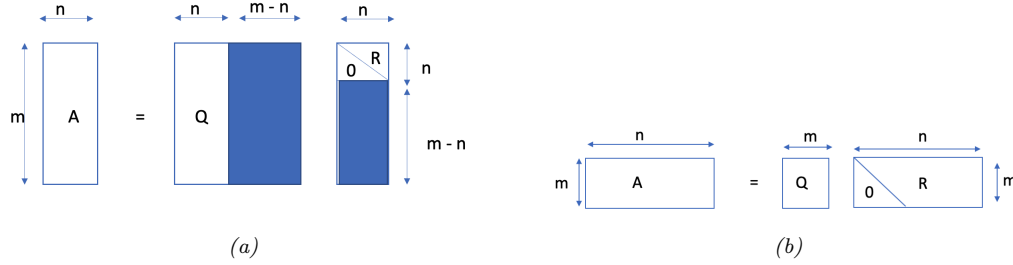


Figure 7.11: Illustration of QR decomposition, $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$ and $\mathbf{R}$ is upper triangular. (a) Tall, skinny matrix. The shaded parts are not computed in the economy-sized version, since they are not needed. (b) Short, wide matrix.

7.6.2 QR decomposition

Suppose we have $\mathbf{A} \in \mathbb{R}^{m \times n}$ representing a set of linearly independent basis vectors (so $m \geq n$), and we want to find a series of orthonormal vectors $\mathbf{q}_1, \mathbf{q}_2, \dots$ that span the successive subspaces of $\text{span}(\mathbf{a}_1)$, $\text{span}(\mathbf{a}_1, \mathbf{a}_2)$, etc. In other words, we want to find vectors $\mathbf{q}_j$ and coefficients r_{ij} such that

$$\begin{pmatrix} | & | & \cdots & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & \cdots & | \end{pmatrix} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \cdots & \mathbf{q}_n \\ | & | & \cdots & | \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \\ & & & r_{nn} \end{pmatrix} \quad (7.204)$$

We can write this

$$\mathbf{a}_1 = r_{11} \mathbf{q}_1 \quad (7.205)$$

$$\mathbf{a}_2 = r_{12} \mathbf{q}_1 + r_{22} \mathbf{q}_2 \quad (7.206)$$

$\vdots$

$$\mathbf{a}_n = r_{1n} \mathbf{q}_1 + \cdots + r_{nn} \mathbf{q}_n \quad (7.207)$$

so we see $\mathbf{q}_1$ spans the space of $\mathbf{a}_1$, and $\mathbf{q}_1$ and $\mathbf{q}_2$ span the space of $\{\mathbf{a}_1, \mathbf{a}_2\}$, etc.

In matrix notation, we have

$$\mathbf{A} = \hat{\mathbf{Q}} \hat{\mathbf{R}} \quad (7.208)$$

where $\hat{\mathbf{Q}}$ is $m \times n$ with orthonormal columns and $\hat{\mathbf{R}}$ is $n \times n$ and upper triangular. This is called a **reduced QR** or **economy sized QR** factorization of $\mathbf{A}$; see Figure 7.11.

A full QR factorization appends an additional $m - n$ orthonormal columns to $\hat{\mathbf{Q}}$ so it becomes a square, orthogonal matrix $\mathbf{Q}$, which satisfies $\mathbf{Q} \mathbf{Q}^T = \mathbf{Q}^T \mathbf{Q} = \mathbf{I}$. Also, we append rows made of zero to $\hat{\mathbf{R}}$ so it becomes an $m \times n$ matrix that is still upper triangular, called $\mathbf{R}$: see Figure 7.11. The zero entries in $\mathbf{R}$ “kill off” the new columns in $\mathbf{Q}$, so the result is the same as $\hat{\mathbf{Q}} \hat{\mathbf{R}}$.

QR decomposition is commonly used to solve systems of linear equations, as we discuss in Section 11.2.2.3.

7.6.3 Cholesky decomposition

Any symmetric positive definite matrix can be factorized as $\mathbf{A} = \mathbf{R}^\top \mathbf{R}$, where $\mathbf{R}$ is upper triangular with real, positive diagonal elements. (This can also be written as $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L} = \mathbf{R}^\top$ is lower triangular.) This is called a **Cholesky factorization** or **matrix square root**. In NumPy, this is implemented by `np.linalg.cholesky`. The computational complexity of this operation is $O(V^3)$, where V is the number of variables, but can be less for sparse matrices. Below we give some applications of this factorization.

7.6.3.1 Application: Sampling from an MVN

The Cholesky decomposition of a covariance matrix can be used to sample from a multivariate Gaussian. Let $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$. We first sample $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, which is easy because it just requires sampling from d separate 1d Gaussians. We then set $\mathbf{y} = \mathbf{L}\mathbf{x} + \boldsymbol{\mu}$. This is valid since

$$\text{Cov}[\mathbf{y}] = \mathbf{L}\text{Cov}[\mathbf{x}]\mathbf{L}^\top = \mathbf{L}\mathbf{I}\mathbf{L}^\top = \boldsymbol{\Sigma} \quad (7.209)$$

See [cholesky_demo.ipynb](#) for some code.

7.7 Solving systems of linear equations *

An important application of linear algebra is the study of systems of linear equations. For example, consider the following set of 3 equations:

$$3x_1 + 2x_2 - x_3 = 1 \quad (7.210)$$

$$2x_1 - 2x_2 + 4x_3 = -2 \quad (7.211)$$

$$-x_1 + \frac{1}{2}x_2 - x_3 = 0 \quad (7.212)$$

We can represent this in matrix-vector form as follows:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (7.213)$$

where

$$\mathbf{A} = \begin{pmatrix} 3 & 2 & -1 \\ 2 & -2 & 4 \\ -1 & \frac{1}{2} & -1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ -2 \\ 0 \end{pmatrix} \quad (7.214)$$

The solution is $\mathbf{x} = [1, -2, -2]$.

In general, if we have m equations and n unknowns, then $\mathbf{A}$ will be a $m \times n$ matrix, and $\mathbf{b}$ will be a $m \times 1$ vector. If $m = n$ (and $\mathbf{A}$ is full rank), there is a single unique solution. If $m < n$, the system is **underdetermined**, so there is not a unique solution. If $m > n$, the system is **overdetermined**, since there are more constraints than unknowns, and not all the lines intersect at the same point. See Figure 7.12 for an illustration. We discuss how to compute solutions in each of these cases below.

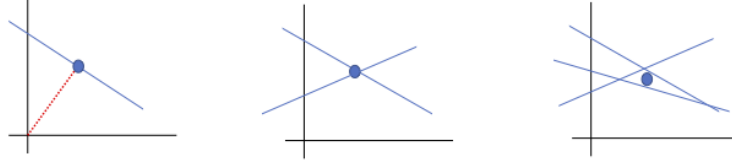


Figure 7.12: Solution of a set of m linear equations in $n = 2$ variables. (a) $m = 1 < n$ so the system is underdetermined. We show the minimal norm solution as a blue circle. (The dotted red line is orthogonal to the line, and its length is the distance to the origin.) (b) $m = n = 2$, so there is a unique solution. (c) $m = 3 > n$, so there is no unique solution. We show the least squares solution.

7.7.1 Solving square systems

In the case where $m = n$, we can solve for $\mathbf{x}$ by computing an LU decomposition, $\mathbf{A} = \mathbf{L}\mathbf{U}$, and then proceeding as follows:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (7.215)$$

$$\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b} \quad (7.216)$$

$$\mathbf{U}\mathbf{x} = \mathbf{L}^{-1}\mathbf{b} \triangleq \mathbf{y} \quad (7.217)$$

$$\mathbf{x} = \mathbf{U}^{-1}\mathbf{y} \quad (7.218)$$

The crucial point is that $\mathbf{L}$ and $\mathbf{U}$ are both triangular matrices, so we can avoid taking matrix inverses, and use a method known as **backsubstitution** instead.

In particular, we can solve $\mathbf{y} = \mathbf{L}^{-1}\mathbf{b}$ without taking inverses as follows. First we write

$$\begin{pmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ & & \ddots & \\ L_{n1} & L_{n2} & \cdots & L_{nn} \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} \quad (7.219)$$

We start by solving $L_{11}y_1 = b_1$ to find y_1 and then substitute this in to solve

$$L_{21}y_1 + L_{22}y_2 = b_2 \quad (7.220)$$

for y_2 . We repeat this recursively. This process is often denoted by the **backslash operator**, $\mathbf{y} = \mathbf{L} \setminus \mathbf{b}$. Once we have $\mathbf{y}$, we can solve $\mathbf{x} = \mathbf{U}^{-1}\mathbf{y}$ using backsubstitution in a similar manner.

7.7.2 Solving underconstrained systems (least norm estimation)

In this section, we consider the underconstrained setting, where $m < n$.³ We assume the rows are linearly independent, so $\mathbf{A}$ is full rank.

3. Our presentation is based in part on lecture notes by Stephen Boyd at <http://ee263.stanford.edu/lectures/min-norm.pdf>.

When $m < n$, there are multiple possible solutions, which have the form

$$\{\mathbf{x} : \mathbf{Ax} = \mathbf{b}\} = \{\mathbf{x}_p + \mathbf{z} : \mathbf{z} \in \text{nullspace}(\mathbf{A})\} \quad (7.221)$$

where $\mathbf{x}_p$ is any particular solution. It is standard to pick the particular solution with minimal ℓ_2 norm, i.e.,

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\text{argmin}} \|\mathbf{x}\|_2^2 \text{ s.t. } \mathbf{Ax} = \mathbf{b} \quad (7.222)$$

We can compute the minimal norm solution using the **right pseudo inverse**:

$$\mathbf{x}_{\text{pinv}} = \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{b} \quad (7.223)$$

(See Section 7.5.3 for more details.)

To see this, suppose $\mathbf{x}$ is some other solution, so $\mathbf{Ax} = \mathbf{b}$, and $\mathbf{A}(\mathbf{x} - \mathbf{x}_{\text{pinv}}) = \mathbf{0}$. Thus

$$(\mathbf{x} - \mathbf{x}_{\text{pinv}})^\top \mathbf{x}_{\text{pinv}} = (\mathbf{x} - \mathbf{x}_{\text{pinv}})^\top \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{b} = (\mathbf{A}(\mathbf{x} - \mathbf{x}_{\text{pinv}}))^\top (\mathbf{AA}^\top)^{-1} \mathbf{b} = 0 \quad (7.224)$$

and hence $(\mathbf{x} - \mathbf{x}_{\text{pinv}}) \perp \mathbf{x}_{\text{pinv}}$. By **Pythagoras's theorem**, the norm of $\mathbf{x}$ is

$$\|\mathbf{x}\|^2 = \|\mathbf{x}_{\text{pinv}} + \mathbf{x} - \mathbf{x}_{\text{pinv}}\|^2 = \|\mathbf{x}_{\text{pinv}}\|^2 + \|\mathbf{x} - \mathbf{x}_{\text{pinv}}\|^2 \geq \|\mathbf{x}_{\text{pinv}}\|^2 \quad (7.225)$$

Thus any solution apart from $\mathbf{x}_{\text{pinv}}$ has larger norm.

We can also solve the constrained optimization problem in Equation (7.222) by minimizing the following unconstrained objective

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{x}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b}) \quad (7.226)$$

From Section 8.5.1, the optimality conditions are

$$\nabla_{\mathbf{x}} \mathcal{L} = 2\mathbf{x} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}, \quad \nabla_{\boldsymbol{\lambda}} \mathcal{L} = \mathbf{Ax} - \mathbf{b} = \mathbf{0} \quad (7.227)$$

From the first condition we have $\mathbf{x} = -\mathbf{A}^\top \boldsymbol{\lambda} / 2$. Substituting into the second we get

$$\mathbf{Ax} = -\frac{1}{2} \mathbf{AA}^\top \boldsymbol{\lambda} = \mathbf{b} \quad (7.228)$$

which implies $\boldsymbol{\lambda} = -2(\mathbf{AA}^\top)^{-1} \mathbf{b}$. Hence $\mathbf{x} = \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{b}$, which is the right pseudo inverse solution.

7.7.3 Solving overconstrained systems (least squares estimation)

If $m > n$, we have an overdetermined solution, which typically does not have an exact solution, but we will try to find the solution that gets as close as possible to satisfying all of the constraints specified by $\mathbf{Ax} = \mathbf{b}$. We can do this by minimizing the following cost function, known as the **least squares objective**.⁴

$$f(\mathbf{x}) = \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|_2^2 \quad (7.233)$$

4. Note that some equation numbers have been skipped. This is intentional. The reason is that I have omitted some erroneous material from an earlier version (described in <https://github.com/probml/pml-book/issues/266>), but want to make sure the equation numbering is consistent across different versions of the book.

Using matrix calculus results from Section 7.8 we have that the gradient is given by

$$\mathbf{g}(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = \mathbf{A}^\top \mathbf{A} \mathbf{x} - \mathbf{A}^\top \mathbf{b} \quad (7.234)$$

The optimum can be found by solving $\mathbf{g}(\mathbf{x}) = \mathbf{0}$. This gives

$$\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b} \quad (7.235)$$

These are known as the **normal equations**, since, at the optimal solution, $\mathbf{b} - \mathbf{A} \mathbf{x}$ is normal (orthogonal) to the range of $\mathbf{A}$, as we explain in Section 11.2.2.2. The corresponding solution $\hat{\mathbf{x}}$ is the **ordinary least squares (OLS)** solution, which is given by

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} \quad (7.236)$$

The quantity $\mathbf{A}^\dagger = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ is the **left pseudo inverse** of the (non-square) matrix $\mathbf{A}$ (see Section 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\mathbf{x}) = \frac{\partial^2}{\partial \mathbf{x}^2} f(\mathbf{x}) = \mathbf{A}^\top \mathbf{A} \quad (7.237)$$

If $\mathbf{A}$ is full rank (so the columns of $\mathbf{A}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\mathbf{v} > \mathbf{0}$, we have

$$\mathbf{v}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v} = (\mathbf{A} \mathbf{v})^\top (\mathbf{A} \mathbf{v}) = \|\mathbf{A} \mathbf{v}\|^2 > 0 \quad (7.238)$$

Hence in the full rank case, the least squares objective has a unique global minimum.

7.8 Matrix calculus

The topic of **calculus** concerns computing “rates of change” of functions as we vary their inputs. It is of vital importance to machine learning, as well as almost every other numerical discipline. In this section, we review some standard results. In some cases, we use some concepts and notation from matrix algebra, which we cover in Chapter 7. For more details on these results from a deep learning perspective, see [PH18].

7.8.1 Derivatives

Consider a scalar-argument function $f : \mathbb{R} \rightarrow \mathbb{R}$. We define its **derivative** at a point x to be the quantity

$$f'(x) \triangleq \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (7.239)$$

assuming the limit exists. This measures how quickly the output changes when we move a small distance in input space away from x (i.e., the “rate of change” of the function). We can interpret $f'(x)$ as the slope of the tangent line at $f(x)$, and hence

$$f(x+h) \approx f(x) + f'(x)h \quad (7.240)$$

for small h .

We can compute a **finite difference** approximation to the derivative by using a finite step size h , as follows:

$$f'(x) \equiv \underbrace{\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h \rightarrow 0} \frac{f(x+h/2) - f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h}}_{\text{backward difference}} \quad (7.241)$$

The smaller the step size h , the better the estimate, although if h is too small, there can be errors due to numerical cancellation.

We can think of differentiation as an operator that maps functions to functions, $D(f) = f'$, where $f'(x)$ computes the derivative at x (assuming the derivative exists at that point). The use of the prime symbol f' to denote the derivative is called **Lagrange notation**. The second derivative function, which measures how quickly the gradient is changing, is denoted by f'' . The n 'th derivative function is denoted $f^{(n)}$.

Alternatively, we can use **Leibniz notation**, in which we denote the function by $y = f(x)$, and its derivative by $\frac{dy}{dx}$ or $\frac{d}{dx}f(x)$. To denote the evaluation of the derivative at a point a , we write $\left. \frac{df}{dx} \right|_{x=a}$.

7.8.2 Gradients

We can extend the notion of derivatives to handle vector-argument functions, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, by defining the **partial derivative** of f with respect to x_i to be

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} \quad (7.242)$$

where $\mathbf{e}_i$ is the i 'th unit vector.

The **gradient** of a function at a point $\mathbf{x}$ is the vector of its partial derivatives:

$$\mathbf{g} = \frac{\partial f}{\partial \mathbf{x}} = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \quad (7.243)$$

To emphasize the point at which the gradient is evaluated, we can write

$$\mathbf{g}(\mathbf{x}^*) \triangleq \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}^*} \quad (7.244)$$

We see that the operator ∇ (pronounced “nabla”) maps a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ to another function $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Since $\mathbf{g}()$ is a vector-valued function, it is known as a **vector field**. By contrast, the derivative function f' is a **scalar field**.

7.8.3 Directional derivative

The **directional derivative** measures how much the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ changes along a direction $\mathbf{v}$ in space. It is defined as follows

$$D_{\mathbf{v}}f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h} \quad (7.245)$$

We can approximate this numerically using 2 function calls to f , regardless of n . By contrast, a numerical approximation to the standard gradient vector takes $n + 1$ calls (or $2n$ if using central differences).

Note that the directional derivative along $\mathbf{v}$ is the scalar product of the gradient $\mathbf{g}$ and the vector $\mathbf{v}$:

$$D_{\mathbf{v}}f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{v} \quad (7.246)$$

7.8.4 Total derivative *

Suppose that some of the arguments to the function depend on each other. Concretely, suppose the function has the form $f(t, x(t), y(t))$. We define the **total derivative** of f wrt t as follows:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt} \quad (7.247)$$

If we multiply both sides by the differential dt , we get the **total differential**

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy \quad (7.248)$$

This measures how much f changes when we change t , both via the direct effect of t on f , but also indirectly, via the effects of t on x and y .

7.8.5 Jacobian

Consider a function that maps a vector to another vector, $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$. The **Jacobian matrix** of this function is an $m \times n$ matrix of partial derivatives:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}^{\top}} \triangleq \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^{\top} \\ \vdots \\ \nabla f_m(\mathbf{x})^{\top} \end{pmatrix} \quad (7.249)$$

Note that we lay out the results in the same orientation as the output $\mathbf{f}$; this is sometimes called numerator layout or the Jacobian formulation.⁵

7.8.5.1 Multiplying Jacobians and vectors

The **Jacobian vector product** or **JVP** is defined to be the operation that corresponds to right-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{v} = \begin{pmatrix} \nabla f_1(\mathbf{x})^{\top} \\ \vdots \\ \nabla f_m(\mathbf{x})^{\top} \end{pmatrix} \mathbf{v} = \begin{pmatrix} \nabla f_1(\mathbf{x})^{\top} \mathbf{v} \\ \vdots \\ \nabla f_m(\mathbf{x})^{\top} \mathbf{v} \end{pmatrix} \quad (7.250)$$

5. For a much more detailed discussion of notation, see https://en.wikipedia.org/wiki/Matrix_calculus.

So we can see that we can approximate this numerically using just 2 calls to $\mathbf{f}$.

The **vector Jacobian product** or **VJP** is defined to be the operation that corresponds to left-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\mathbf{u} \in \mathbb{R}^m$:

$$\mathbf{u}^\top \mathbf{J}_f(\mathbf{x}) = \mathbf{u}^\top \left(\frac{\partial \mathbf{f}}{\partial x_1}, \dots, \frac{\partial \mathbf{f}}{\partial x_n} \right) = \left(\mathbf{u} \cdot \frac{\partial \mathbf{f}}{\partial x_1}, \dots, \mathbf{u} \cdot \frac{\partial \mathbf{f}}{\partial x_n} \right) \quad (7.251)$$

The JVP is more efficient if $m \geq n$, and the VJP is more efficient if $m \leq n$. See Section 13.3 for details on how this can be used to perform automatic differentiation in a computation graph such as a DNN.

7.8.5.2 Jacobian of a composition

Sometimes it is useful to take the Jacobian of the composition of two functions. Let $h(\mathbf{x}) = g(f(\mathbf{x}))$. By the chain rule of calculus, we have

$$\mathbf{J}_h(\mathbf{x}) = \mathbf{J}_g(f(\mathbf{x}))\mathbf{J}_f(\mathbf{x}) \quad (7.252)$$

For example, suppose $f : \mathbb{R} \rightarrow \mathbb{R}^2$ and $g : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. We have

$$\frac{\partial \mathbf{g}}{\partial x} = \left(\frac{\partial}{\partial x} g_1(f_1(x), f_2(x)), \frac{\partial}{\partial x} g_2(f_1(x), f_2(x)) \right) = \begin{pmatrix} \frac{\partial g_1}{\partial f_1} \frac{\partial f_1}{\partial x} + \frac{\partial g_1}{\partial f_2} \frac{\partial f_2}{\partial x} \\ \frac{\partial g_2}{\partial f_1} \frac{\partial f_1}{\partial x} + \frac{\partial g_2}{\partial f_2} \frac{\partial f_2}{\partial x} \end{pmatrix} \quad (7.253)$$

$$= \frac{\partial \mathbf{g}}{\partial \mathbf{f}^\top} \frac{\partial \mathbf{f}}{\partial x} = \begin{pmatrix} \frac{\partial g_1}{\partial f_1} & \frac{\partial g_1}{\partial f_2} \\ \frac{\partial g_2}{\partial f_1} & \frac{\partial g_2}{\partial f_2} \end{pmatrix} \begin{pmatrix} \frac{\partial f_1}{\partial x} \\ \frac{\partial f_2}{\partial x} \end{pmatrix} \quad (7.254)$$

7.8.6 Hessian

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that is twice differentiable, we define the **Hessian matrix** as the (symmetric) $n \times n$ matrix of second partial derivatives:

$$\mathbf{H}_f = \frac{\partial^2 f}{\partial x^2} = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix} \quad (7.255)$$

We see that the Hessian is the Jacobian of the gradient.

7.8.7 Gradients of commonly used functions

In this section, we list without proof the gradients of certain widely used functions.

7.8.7.1 Functions that map scalars to scalars

Consider a differentiable function $f : \mathbb{R} \rightarrow \mathbb{R}$. Here are some useful identities from scalar calculus, which you should already be familiar with.

“Probabilistic Machine Learning: An Introduction”. Online version. June 26, 2024

$$\frac{d}{dx} cx^n = cnx^{n-1} \quad (7.256)$$

$$\frac{d}{dx} \log(x) = 1/x \quad (7.257)$$

$$\frac{d}{dx} \exp(x) = \exp(x) \quad (7.258)$$

$$\frac{d}{dx} [f(x) + g(x)] = \frac{df(x)}{dx} + \frac{dg(x)}{dx} \quad (7.259)$$

$$\frac{d}{dx} [f(x)g(x)] = f(x)\frac{dg(x)}{dx} + g(x)\frac{df(x)}{dx} \quad (7.260)$$

$$\frac{d}{dx} f(u(x)) = \frac{du}{dx} \frac{df(u)}{du} \quad (7.261)$$

Equation (7.261) is known as the **chain rule of calculus**.

7.8.7.2 Functions that map vectors to scalars

Consider a differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Here are some useful identities:⁶

$$\frac{\partial(\mathbf{a}^\top \mathbf{x})}{\partial \mathbf{x}} = \mathbf{a} \quad (7.262)$$

$$\frac{\partial(\mathbf{b}^\top \mathbf{A} \mathbf{x})}{\partial \mathbf{x}} = \mathbf{A}^\top \mathbf{b} \quad (7.263)$$

$$\frac{\partial(\mathbf{x}^\top \mathbf{A} \mathbf{x})}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x} \quad (7.264)$$

It is fairly easy to prove these identities by expanding out the quadratic form, and applying scalar calculus.

7.8.7.3 Functions that map matrices to scalars

Consider a function $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ which maps a matrix to a scalar. We are using the following natural layout for the derivative matrix:

$$\frac{\partial f}{\partial \mathbf{X}} = \begin{pmatrix} \frac{\partial f}{\partial x_{11}} & \cdots & \frac{\partial f}{\partial x_{1n}} \\ \vdots & & \vdots \\ \frac{\partial f}{\partial x_{m1}} & \cdots & \frac{\partial f}{\partial x_{mn}} \end{pmatrix} \quad (7.265)$$

Below are some useful identities.

6. Some of the identities are taken from the list at <http://www.cs.nyu.edu/~roweis/notes/matrixid.pdf>.

Identities involving quadratic forms

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}(\mathbf{a}^\top \mathbf{X} \mathbf{b}) = \mathbf{a} \mathbf{b}^\top \quad (7.266)$$

$$\frac{\partial}{\partial \mathbf{X}}(\mathbf{a}^\top \mathbf{X}^\top \mathbf{b}) = \mathbf{b} \mathbf{a}^\top \quad (7.267)$$

Identities involving matrix trace

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}} \text{tr}(\mathbf{A} \mathbf{X} \mathbf{B}) = \mathbf{A}^\top \mathbf{B}^\top \quad (7.268)$$

$$\frac{\partial}{\partial \mathbf{X}} \text{tr}(\mathbf{X}^\top \mathbf{A}) = \mathbf{A} \quad (7.269)$$

$$\frac{\partial}{\partial \mathbf{X}} \text{tr}(\mathbf{X}^{-1} \mathbf{A}) = -\mathbf{X}^{-\top} \mathbf{A}^\top \mathbf{X}^{-\top} \quad (7.270)$$

$$\frac{\partial}{\partial \mathbf{X}} \text{tr}(\mathbf{X}^\top \mathbf{A} \mathbf{X}) = (\mathbf{A} + \mathbf{A}^\top) \mathbf{X} \quad (7.271)$$

Identities involving matrix determinant

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}} \det(\mathbf{A} \mathbf{X} \mathbf{B}) = \det(\mathbf{A} \mathbf{X} \mathbf{B}) \mathbf{X}^{-\top} \quad (7.272)$$

$$\frac{\partial}{\partial \mathbf{X}} \log(\det(\mathbf{X})) = \mathbf{X}^{-\top} \quad (7.273)$$

7.9 Exercises

Exercise 7.1 [Orthogonal matrices]

a. A rotation in 3d by angle α about the z axis is given by the following matrix:

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (7.274)$$

Prove that $\mathbf{R}$ is an orthogonal matrix, i.e., $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, for any α .

b. What is the only eigenvector $\mathbf{v}$ of $\mathbf{R}$ with an eigenvalue of 1.0 and of unit norm (i.e., $\|\mathbf{v}\|^2 = 1$)? (Your answer should be the same for any α .) Hint: think about the geometrical interpretation of eigenvectors.

Exercise 7.2 [Eigenvectors by hand †]

Find the eigenvalues and eigenvectors of the following matrix

$$\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \quad (7.275)$$

Compute your result by hand and check it with Python.